

15. Bölüm

Şablonlar

15.1 Şablon Nedir?

Şablonlar (template), veri tiplerine bağlı kalmadan çalışan fonksiyonlar, sınıflar, algoritmalar yazmak için jenerik programlama (**generic programming**) ile oluşturulan yapılardır.

Jenerik programlama, kodun belirli bir veri tipine (**int**, **float**, **string** vb.) bağlı kalmadan, veri tipinden bağımsız bir şekilde yazılmasına olanak tanıyan bir yazılım geliştirme yaklaşımıdır. En temel amacı, aynı mantığa sahip işlemleri farklı veri tipleri için tekrar tekrar yazmak yerine, bir kez **şablon (template)** olarak hazırlayıp her tipte güvenle kullanabilmektir. Yani;

- ◊ Bir kez yazılan bir algoritma (örneğin sıralama veya arama), hem sayılar hem de metinler için çalışabilir. Böylece aynı kodu yeniden kullanılabiliriz (**reusing**).
- ◊ Kod derleme zamanında (**compile-time**) kontrol edilir. Hatalar program çalışmadan yakalanır. Böylece tip güvenliği (**type safety**) sağlanmış olur.
- ◊ Çalışma zamanında (**run-time**) bir tip kontrolü yapılmaz; derleyici her tip için özel bir kod üretir, bu yüzden çok hızlıdır. Böylece performans yakalamış oluruz.

Jenerik programlamada bir fonksiyon veya sınıf yazarken, tip kısmına gerçek bir isim (örneğin **int**) yerine bir yer tutucu (**placeholder**) koyarsınız. Genellikle bu yer tutucu için **T** harfi kullanılır.

Örnek olarak iki değeri karşılaştırıp büyüğünü bulan bir jenerik fonksiyon düşünelim. Jenerik olmayan kodda her veri tipi için bir fonksiyon yazmayı gerektirir;

```
intMax(int a, int b) {  
    //...  
}  
doubleMax(double a, double b) {  
    //...  
}
```

Jenerik olan kodda ise tek bir şablon fonksiyon yazarsınız.

```
template <typename T> T Max(T a, T b) {  
    return (a>=b)?a:b;  
}
```

Derleyici, siz içine ne koyarsanız (**int** veya **double**) ona uygun gerçek kodu arka planda sizin için oluşturur. Eğer jenerik programlama olmasaydı, her yeni veri tipi için aynı fonksiyonları baştan yazmak zorunda kalırdık. Bu da hem çok fazla **kod kalabalığına (code bloat)** hem de hata yapma riskinin artmasına neden olurdu.

15.2 Foksiyon Şablonları

Jenerik programlamanın temel taşı olan fonksiyon şablonları (**function templates**), aynı mantığı yeniden yazmadan bir fonksiyonun farklı veri tipleri üzerinde çalışmasını sağlayan bir fonksiyon planı tanımlar. Bu plan aşağıdaki şekilde tanımlanır;

```
template <typename değişken-kimliği> foksiyon-bildirimi;
```

Aşağıdaki kodda, iki değerini yerini değiştiren (**swap**) bir fonksiyonu hem **int** hem **double** hem de **string** için tek bir şablonla nasıl çalıştırdığımızı görebilirsiniz.

```
#include <iostream>  
#include <string>  
// 'typename T' ifadesi, T'nin herhangi bir veri tipi olabileceğini belirtir.  
template <typename T>  
void DegerDegistir(T &a, T &b) {  
    T gecici = a;  
    a = b;  
    b = gecici;
```

```

}
int main() {
    // 1. Tam Sayılarla Kullanım
    int x = 10, y = 20;
    DegerDegistir(x, y); // Derleyici T'yi 'int' olarak anlar.
    std::cout << "Tam Sayilar: x=" << x << ", y=" << y << std::endl;

    // 2. Ondalıklı Sayılarla Kullanım
    double d1 = 1.5, d2 = 9.9;
    DegerDegistir(d1, d2); // Derleyici T'yi 'double' olarak anlar.
    std::cout << "Ondalikli: d1=" << d1 << ", d2=" << d2 << std::endl;

    // 3. Metinlerle Kullanım
    std::string s1 = "Dunya", s2 = "Merhaba";
    DegerDegistir(s1, s2); // Derleyici T'yi 'string' olarak anlar.
    std::cout << "Metinler: s1=" << s1 << ", s2=" << s2 << std::endl;
}
/*Program Çıktısı:
Tam Sayilar: x=20, y=10
Ondalikli: d1=9.9, d2=1.5
Metinler: s1=Merhaba, s2=Dunya
*/

```

15.3 Değişken Sayıda Argüman Alabilen Fonksiyon Şablonları

Değişken sayıda argüman alabilen fonksiyon şablonları (**variadic function template**), herhangi bir sayıda (sıfır veya daha fazla) argüman alabilen sınıf veya fonksiyon şablonlarıdır. C++ dilinde şablonlar, yalnızca bildirim sırasında belirtilmesi gereken sabit sayıda parametreye sahip olabilir. Ancak *Douglas Gregor* ve *Jaakko Järvi* tarafından ortaya koyulan **değişken sayıda argüman alabilen şablonlar** (**variadic template**) bu sorunun üstesinden gelmeye yardımcı olur.

C++11 ile hayatımıza giren ve bir fonksiyona sınırsız sayıda ve farklı türde argüman göndermemize olanak tanıyan oldukça güçlü bir jenerik programlama özelliğidir. Değişken sayıda argüman alabilen şablonlar (**variadic şablonlar**) bir **parametre paketi** (**parameter pack**) kullanarak her şeyi kabul edebilir. Temel söz diziminde üç nokta (...) karakterleri bulunur;

```

template <typename... Args>
void fonksiyon(Args... args) {
    // Args: Şablon parametre paketi (tipler)
    // args: Fonksiyon parametre paketi (değerler)
}

```

Bu söz diziminde; **typename... Args** demek buraya sıfır veya daha fazla tip gelebilir demektir. **Args... args** ise bu tiplerden oluşan paketlenmiş değerleri al demektir.

Değişken sayıda argüman alabilen şablonlar (**variadic şablonlar**), 7.8 başlığında anlatılan özyinelemeli mantıkla çalışır. Paketin içindeki ilk elemanı alırız, geri kalan paketi tekrar aynı fonksiyona göndeririz. Paket boşaldığında ise durmak için bir **sonlandırıcı fonksiyon** (**base case**) yazarız. Örnek olarak argüman sayısı belli olmayan bir yazdır fonksiyonunu örnek verebiliriz;

```

#include <iostream>
// 1. Sonlandırıcı (Base Case): Paket boşaldığında bu çalışır.
void Yazdir() {
    std::cout << std::endl;
}
// 2. Variadic Şablon:
template <typename T, typename... Args>
void Yazdir(T ilk, Args... geriKalan) {
    std::cout << ilk << " "; // İlk elemanı yazdır

    // Paketi "açarak" (unpacking) geri kalanını tekrar gönderiyoruz
    Yazdir(geriKalan...);
}
int main() {
    Yazdir(1, 2.5, "Merhaba", 'A', 42);
}

```

```
// Adım 1: 1 yazdırılır, (2.5, "Merhaba", 'A', 42) gönderilir.
// Adım 2: 2.5 yazdırılır, ("Merhaba", 'A', 42) gönderilir.
// ... en son boş paket Yazdir() fonksiyonuna düşer.
}
/*Program Çıktısı:
1 2.5 Merhaba A 42
*/
```

Argüman paketinin içinde kaç argümanın olduğunu **sizeof... işlecini** (**sizeof... operator**) kullanarak öğrenebiliriz. Ayrıca C++17 öncesinde yukarıdaki gibi özyinelemeli (**recursive**) yazmak zorundaydık. C++17 ile gelen **katlama ifadeleri** (**fold expressions**) sayesinde artık döngüye veya **sonlandırıcı fonksiyona** (**base case**) gerek kalmadan paketi tek satırda açabiliriz:

```
#include <iostream>
template<typename... Args>
void YeniYazdir(Args... args) {
    std::cout << "Gelen argüman sayısı: " << sizeof...(args) << std::endl;
    // Tüm argümanların arasına << koyarak tek seferde yazdır (Fold Expression)
    (std::cout << ... << args) << std::endl;
}
int main() {
    YeniYazdir(1, 2.5, "Merhaba", 'A', 42);
    // Adım 1: 1 yazdırılır, (2.5, "Merhaba", 'A', 42) gönderilir.
    // Adım 2: 2.5 yazdırılır, ("Merhaba", 'A', 42) gönderilir.
    // ... en son boş paket Yazdir() fonksiyonuna düşer.
}
/*Program Çıktısı:
Gelen argüman sayısı: 5
12.5MerhabaA42
*/
```

15.4 Katlama İfadeleri

Katlama ifadeleri (**fold expressions**), C++17 standardı ile gelen ve değişken sayıda parametre alan şablonlar ile çalışmayı inanılmaz derecede kolaylaştıran bir özelliktir. C++11 ve C++14'te, bir parametre paketindeki (**parameter pack**) tüm elemanları toplamak veya çarpmak için özyinelemeli (**recursive**) fonksiyonlar yazmak zorundaydık. Katlama ifadeleri sayesinde artık tek bir satırda bu işlemi yapabiliyoruz.

Parametre Paketini katlamaya şu örnek verilebilir; Örneğin, elinizde 1, 2, 3, 4 gibi bir paket varsa, + işleciyle bunu (((1 + 2) + 3) + 4) haline getirir. Dört Farklı Katlama Türü vardır. Aşağıdaki tabloda **args** parametre paketini, **op** ise + ve * gibi ikili (iki işleneni olan) işlemleri temsil eder;

Katlama Türü	Yazım Şekli	Açılımı (Örnek)
Unary Right Fold	(args op ...)	(a1 op (a2 op (a3 op a4)))
Unary Left Fold	(... op args)	((a1 op a2) op a3) op a4
Binary Right Fold	(args op ... op init)	(a1 op (a2 op (a3 op (a4 op init))))
Binary Left Fold	(init op ... op args)	((((init op a1) op a2) op a3) op a4)

Tablo 15.1: Katlama Türleri

Katlama ifadeleri şunlar için kullanmalıyız;

- ◊ **Kod Kısalığı:** Özyinelemeli fonksiyon şablonları yazmanıza ve "durma koşulu" (**base case**) belirlemenize gerek kalmaz.
- ◊ **Derleme Hızı:** Derleyici bu ifadeleri doğrudan açtığı için özyinelemeli şablonlara göre daha hızlı derlenir.
- ◊ **Okunabilirlik:** Niyetinizi (tüm paketi bir operatörle birleştirmek) koda çok daha net yansıtır.

Katlama İfadelerini Destekleyen İşleçler

Neredeyse tüm ikili operatörler desteklenir: + - * / % ^ & | = < > << >> == != <= >= && || , . * ->*

Değişken sayıda sayıyı toplama **Unary Left Fold** katlama ile eskiden sayfalarca süren bu işlem artık tek satırdır;

```
#include <iostream>
```

```
template<typename... Args>
auto toplam(Args... args) {
    return (... + args); // Unary Left Fold
}
int main() {
    std::cout << toplam(1, 2, 3, 4, 5); // Çıktı: 15
    return 0;
}
```

Hepsini ekrana yazdırmak için virgül işleci (**comma operator**) katlama ifadeleri ile kullanıldığında, her bir argüman için bir işlem yapılmasını sağlar:

```
#include <iostream>
template<typename... Args>
void hepsiniYazdir(Args... args) {
    (std::cout << ... << args) << std::endl; // Binary Left Fold (std::cout başlangıç değeri)
}
int main() {
    hepsiniYazdir("C++ ", 17, " Fold ", "Expressions ", "Harika!");
}
```

Aşağıdaki örnekte, gönderilen tüm argümanların **true** olup olmadığını kontrol eden bir fonksiyon örneği vardır;

```
#include <iostream>
#include <vector>
// Hepsi doğru mu? (Logical AND Fold)
template<typename... Args>
bool hepsiDogruMu(Args... args) {
    return (... && args); // Unary Left Fold: (a1 && a2 && a3 ...)
}
// En az biri doğru mu? (Logical OR Fold)
template<typename... Args>
bool enAzBiriDogruMu(Args... args) {
    return (... || args); // Unary Left Fold: (a1 || a2 || a3 ...)
}
int main() {
    bool sonuc1 = hepsiDogruMu(true, 1, (5 > 2), true);
    std::cout << "Hepsi dogru mu? " << std::boolalpha << sonuc1 << std::endl; // true
    bool sonuc2 = hepsiDogruMu(true, 0, true);
    std::cout << "Hepsi dogru mu? " << sonuc2 << std::endl; // false (cunku 0, false kabul edilir)
    bool sonuc3 = enAzBiriDogruMu(false, 0, (10 < 2), true);
    std::cout << "En az biri dogru mu? " << sonuc3 << std::endl; // true
}
```

Katlama için iki durum meydana gelebilir;

1. **Kısa Devre Yapma (short-circuiting)**: Katlama ifadeleri, mantıksal operatörlerin doğal davranışını korur. Örneğin (... && args) ifadesinde, paket içindeki ilk eleman **false** ise, derleyici diğer elemanlara bakmaz ve sonucu anında **false** döner. Bu performans için kritiktir.
2. **Boş Parametre Paketi Durumu**: Eğer fonksiyona hiç parametre göndermezseniz (**hepsiDogruMu()**), C++ standartlarına göre mantıksal katlamalar varsayılan değerler döner: **&&** (AND) için boş paket **true** döner. **||** (OR) için boş paket **false** döner.
3. **,** (virgül) için boş paket Geçersizdir (**void**).

15.5 Sınıf Şablonları

Sınıf şablonları (class templates), fonksiyon şablonlarında olduğu gibi, bir sınıfı belirli bir veri tipine bağlı kalmadan, tipten bağımsız bir kalıp olarak tanımlamanızı sağlar. Bir sınıf şablonu yazdığınızda, aslında bir sınıf değil, o sınıfın nasıl oluşturulacağına dair bir reçete yazmış olursunuz. Temel söz dizimi aşağıda verilmiştir;

```
template <typename T>
class SablonSinif {
private:
    T icerik; // T tipinde bir veri saklar
public:
```

```

SablonSinif(T pIcerik) : icerik(pIcerik) {}
T getIcerik() const {
    return icerik;
}
void setIcerik(T pYeniIcerik) {
    icerik = pYeniIcerik;
}
};

```

Bir sınıf şablonundan nesne üretirken, hangi veri tipini şablona parametre vereceğinizi küçük ve büyük karakterleri (< >) içinde belirtirsiniz.

```

#include <iostream>

template <typename T>
class SablonSinif {
private:
    T icerik; // T tipinde bir veri saklar
public:
    SablonSinif(T pIcerik) : icerik(pIcerik) {}
    T getIcerik() const {
        return icerik;
    }
    void setIcerik(T pYeniIcerik) {
        icerik = pYeniIcerik;
    }
};

int main() {
    // T yerine 'int' geçiyoruz.
    SablonSinif<int> tamSayiSinifi(42);
    // T yerine 'string' geçiyoruz.
    SablonSinif<std::string> metinSinifi("Merhaba C++");

    std::cout << tamSayiSinifi.getIcerik() << std::endl;
    std::cout << metinSinifi.getIcerik() << std::endl;
}

/*Program Çıktısı:
42
Merhaba C++
*/

```

Eğer bir şablon sınıfın metodunu sınıfın dışında tanımlamak isterseniz, her seferinde şablon ifadesini tekrar etmeniz gerekir:

```

#include <iostream>

template <typename T>
class SablonSinif {
private:
    T icerik; // T tipinde bir veri saklar
public:
    SablonSinif(T pIcerik) : icerik(pIcerik) {}
    T getIcerik() const;
    void setIcerik(T pYeniIcerik);
};

template <typename T>
T SablonSinif<T>::getIcerik() const {
    return icerik;
}

template <typename T>
void SablonSinif<T>::setIcerik(T pYeniIcerik) {
    icerik = pYeniIcerik;
}

int main() {

```

```
// T yerine 'int' geçiyoruz
SablonSinif<int> tamSayiSinifi(42);
// T yerine 'string' geçiyoruz
SablonSinif<std::string> metinSinifi("Merhaba C++");

std::cout << tamSayiSinifi.getIcerik() << std::endl;
std::cout << metinSinifi.getIcerik() << std::endl;
}
/*Program Çıktısı:
42
Merhaba C++
*/
```

Sınıf şablonunda sadece bir veri tipi değil, birden fazla farklı veri tipini de aynı anda kullanabilirsiniz. Örnek olarak `Cift<T1, T2>` şablonu aşağıda verilmiştir:

```
#include <iostream>
#include <string>
template <typename T1, typename T2>
class Cift {
public:
    T1 birincil;
    T2 ikincil;
    // Yapıcı (Constructor)
    Cift(T1 a, T2 b) : birincil(a), ikincil(b) {}
    void ciftYaz() {
        // T1 ve T2 tiptir, değişkenler ise 'birincil' ve 'ikincil'dir.
        std::cout << birincil << ": " << ikincil << std::endl;
    }
};
int main() {
    // Örnek 1: Yaş bilgisi (string ve int)
    Cift<std::string, int> yasVerisi("Yasim", 45);
    yasVerisi.ciftYaz();
    // Örnek 2: Cinsiyet bilgisi (string ve string)
    Cift<std::string, std::string> cinsiyetVerisi("Cinsiyetim", "ERKEK");
    cinsiyetVerisi.ciftYaz();
}
/*Program Çıktısı:
Yasim: 45
Cinsiyetim: ERKEK
*/
```

15.6 Faydalı Şablonlar

C++ dünyasındaki `<utility>` başlığı "isviçre çakısı" rolüne sahiptir. Bu başlık büyük veri yapıları sunmaz; bunun yerine algoritmaların ve sınıfların birbiriyle konuşmasını sağlayan, performansı artıran küçük ama hayati araçlar sunar. Bu araçlar tek başlarına birer dev olmasalar da, taşıma semantiği (**move semantics**) ve genel programlama (**generic programming**) gibi modern C++ kavramlarının temelini oluştururlar.

§15.6.1 `std::pair`

En eski ve en temel bileşenlerden biridir. İki farklı veri tipini tek bir birim olarak paketlemenizi sağlar. Genellikle bir fonksiyondan iki değer döndürmek gerektiğinde tercih edilir.

```
#include <iostream>
#include <utility>
#include <string>
int main() {
    // İkilinin oluşturulması
    std::pair<int, std::string> ogrenci(101, "Ali Yilmaz");
    // Elemanlara erişim (.first ve .second)
    std::cout << "ID: " << ogrenci.first << " Isim: " << ogrenci.second << std::endl;
    // std::make_pair ile tip belirtmeden oluşturma
    auto urun = std::make_pair("Defter", 25.50);
}
```

```
}
```

§15.6.2 std::move

C++11 ile gelen en devrimsel araçlardan biridir. Bir nesnenin içeriğini kopyalamak yerine, o içeriği başka bir nesneye "taşınması" sağlar. Özellikle büyük vektörler veya metinlerle çalışırken performansı dramatik şekilde artırır. `std::move()` aslında bir şeyi taşımaz; sadece nesneyi "taşınabilir" (**rvalue**) olarak işaretler. Asıl taşıma işlemini sınıfın taşıma yapıcısı (**move constructor**) metodu yapar.

```
#include <vector>
#include <string>
#include <utility>
std::string metin = "Bu cok büyük bir veri kumesidir...";
std::vector<std::string> liste;
// Kopyalama yapmak yerine veriyi listeye taşıyoruz:
liste.push_back(std::move(metin));
// Bu noktadan sonra 'metin' değişkeni boştur veya belirsiz bir durumdadır.
```

§15.6.3 std::swap

İki değişkenin değerlerini yer değiştirmek için kullanılır. `<utility>` içindeki `std::swap()`, eğer nesne destekliyse otomatik olarak taşıma semantiğini kullanır, bu da onu son derece hızlı kılar.

```
int x = 10, y = 20;
std::swap(x, y); // x=20, y=10 olur.
```

§15.6.4 std::forward

Şablon (**template**) yazarken, bir fonksiyonun aldığı parametreleri başka bir fonksiyona "olduğu gibi" (lvalue ise lvalue, rvalue ise rvalue olarak) aktarmak için kullanılır. Özellikle jenerik programlama (**generic programming**) yaparken vazgeçilmezdir.

```
template <typename T>
void araciFonksiyon(T&& arg) {
    // arg'ı gerçekİslem fonksiyonuna geldiği haliyle aktarır
    gerçekİslem(std::forward<T>(arg));
}
```

§15.6.5 Değişmez Değerler ve Diğerleri

C++17 ile gelen `std::as_const`, bir değişkeni geçici olarak salt okunur (**const**) hale getirir. Özellikle yanlışlıkla veri değiştirilmesini önlemek için fonksiyonlara parametre gönderirken kullanılır.

C++14 ile birlikte gelen `std::exchange`, bir değişkenin değerini yenisiyle değiştirirken, eski değerini döndürür. Özellikle yıkıcı (**destructor**) veya taşıma atama işleci (**move assignment**) yazarken çok kullanışlıdır.

```
int eski_deger = std::exchange(guncel_deger, 0); // guncel_deger'i 0 yapar, eskiyi döndürür.
```

15.7 Akıllı Göstericiler

§15.7.1 Kaynak Edinimi İlk Değer Atamasıdır! Prensibi

Modern C++'ın bel kemiği sayılabilecek en önemli konulardan birine geldik. RAII (**Resource Acquisition Is Initialization**), yani "Kaynak Edinimi İlk Değer Atamasıdır" prensibi, C++'ı diğer dillerden ayıran ve bellek yönetimini güvenli kılan temel felsefedir. RAII, bir kaynağın (bellek, dosya göstericisi, ağ bağlantısı, mutex kiliti vb.) ömrünü bir nesnenin ömrüne bağlama tekniğidir. Bu prensip sayesinde C++'ta **çöp toplayıcı** (**garbage collector**) mekanizmasına ihtiyaç duyulmadan bellek sızıntıları önlenir.

RAII Prensibi

RAII'nin temel işleyişi yapıcı (**constructor**) ve yıkıcıların (**destructor**) gücüne dayanır. İki ana adıma dayanır: Yapıcıda, kaynak (örneğin **new** ile ayrılan bellek veya açılan bir dosya) nesne oluşturulduğu anda edinilir. Yıkıcıda ise nesne kapsam (**scope**) dışına çıktığı anda kaynak otomatik olarak serbest bırakılır.

Geleneksel programlamada en büyük risk, kaynağı serbest bırakmayı unutmaktır;

```
void riskliFonksiyon() {
    FILE* dosya = fopen("data.txt", "r");
    // ... işlemler yapılırken bir hata oluştu ve 'return' edildi ...
```



```

    if (hataOlustu) return; // Dosya kapatılmadı! (Sızıntı)

    fclose(dosya);
}

```

RAII Çözümü, **kendi kendini temizleyen sınıf yazmaktır**. Aynı kodu RAII ile aynı işlemi güvenli hale getirelim:

```

#include <iostream>
#include <fstream>
#include <string>
class DosyaYonetici {
private:
    std::ifstream dosya;
public:
    // Kaynak yapıcıda edinilir (Acquisition is Initialization)
    DosyaYonetici(std::string dosyaAdi) {
        dosya.open(dosyaAdi);
        std::cout << "Dosya acildi.\n";
    }
    // Kaynak yıkıcıda serbest bırakılır
    ~DosyaYonetici() {
        if (dosya.is_open()) {
            dosya.close();
            std::cout << "Dosya otomatik kapatildi.\n";
        }
    }
    void oku() { /* ... okuma işlemleri ... */ }
};
void güvenliFonksiyon() {
    DosyaYonetici yönetici("test.txt");
    yönetici.oku();
    // Fonksiyon burada bitse de, hata fırlatsa da 'yönetici' nesnesi
    // kapsam dışı kalacağı için yıkıcısı çalışır ve dosya kapanır.
}

```

C++'ta bir fonksiyon bittiğinde, o fonksiyon içindeki yerel nesnelerin yıkıcıları garantili olarak çağrılır. RAII bu garantiyi kullanarak kaynakları temizler.

Modern C++'ta kendi RAII sınıflarınızı yazabileceğiniz gibi, Standart Kütüphane (STL) zaten birçok hazır RAII aracı sunar:

- ♦ **Akıllı göstericiler** (`std::unique_ptr`, `std::shared_ptr`): Dinamik belleği (**heap**) RAII prensibiyle yönetir. `delete` yazmanıza gerek kalmaz.
- ♦ **Konteynerler** (`std::vector`, `std::string`): İçlerindeki dizileri otomatik büyütür ve yok ederler.
- ♦ **Kilit Yönetimi** (`std::lock_guard`): Multi-thread programlamada bir mutex'i kilitler ve kapsam bitince kilidi otomatik açar.

C++ dünyasında **akıllı göstericiler** (**smart pointers**), bellek yönetimini otomatik olarak yapan ve **bellek sızıntısı** (**memory leak**) gibi korkulu rüyaları ortadan kaldıran modern bir sarmalayıcı (**wrapper**) şablonlardır. Geleneksel C++ dilinde `new` ile tahsis edilen bir belleği `delete` ile silmeyi unutmak programın şişmesine neden olurken; Akıllı göstericiler, nesnenin ömrü dolduğunda belleği kendi kendine serbest bırakır. Göstericilerin işlendiği 9 başlığında, geleneksel göstericilerde şu üç temel risk vardır:

- ♦ Bellek Sızıntısı: `delete` satırı unutulursa bellek geri verilmez.
- ♦ Erken Silme: Bellek silindikten sonra hala o adrese erişmeye çalışan sarkan göstericiler (**dangling pointer**) programı çökertir.
- ♦ Çift Silme: Aynı adresi yanlışlıkla iki kez serbest bırakmaya (**double free**) çalışmak.

Göstericiler `<memory>` başlık dosyasında yer alır. Kullanmak için projemize dahil etmemiz gerekir. Üç ana akıllı gösterici bulunur ve aşağıda başlıklar halinde verilmiştir.

§15.7.2 `std::unique_ptr` ile Tekil Sahiplik

`std::unique_ptr` ile **Akıllı gösterici nesnesinin sadece tek bir sahibi olabilir**. Kopyalanamaz, sadece `std::move` fonksiyonu ile taşınabilir. Akıllı gösterici nesnenin ömrü tek bir kapsam (**scope**) içindeyse veya sahiplik net bir şekilde devredilecekse kullanılır. Bildiğimiz göstericiler kadar hızlıdır, ek yük getirmez.


```
#include <iostream>
#include <memory> // Akıllı göstericiler için
int main() {
    std::unique_ptr<int> ptr1 = std::make_unique<int>(10);
    // std::unique_ptr<int> p2 = p1; // HATA! Kopyalanamaz.
    std::unique_ptr<int> ptr2 = std::move(ptr1); // p1 artık boş, p2 sahibi.
}
```

§15.7.3 std::shared_ptr ile Paylaşılan Sahiplik

`std::shared_ptr` ile bir gösterici nesnenin birden fazla sahibi olabilir. Arka planda bir **referans sayacı** (**reference counter**) tutar. Yeni bir `shared_ptr` nesneye bağlandığında sayaç artar. Bir işaretçi yok olduğunda sayaç azalır. Sayaç sıfır olduğunda bellek temizlenir. Sayaç yönetimi nedeniyle çok küçük bir çalışma zamanı yükü vardır.

```
#include <iostream>
#include <memory> // Akıllı göstericiler için
int main() {
    auto s1 = std::make_shared<int>(100);
    {
        auto s2 = s1; // Sayaç 2 oldu.
        auto s3 = s1; // Sayaç 3 oldu.
    } // s2,s3 kapsam dışı, sayaç 1'e düştü.
    // main bitince sayaç 0 olur ve bellek silinir.
}
```

§15.7.4 std::weak_ptr ile Zayıf Takipçi

`std::shared_ptr` tarafından yönetilen bir nesneye erişim sağlar ancak sayacı artırmaz. **Döngüsel bağımlılık** (**circular dependency**) sorununu çözmek için kullanılır. İki nesne birbirini `shared_ptr` ile tutarsa sayaç asla sıfırlanmaz; `weak_ptr` bu düğümü çözer. Aşağıdaki senaryoda bir **Anne** ve bir **Cocuk** nesnesi var. Anne çocuğunu, çocuk da annesini `shared_ptr` ile işaret ediyor. Döngüsel bağımlılık oluşmuştur.

```
#include <iostream>
#include <memory>
struct Cocuk; // Forward declaration: Cocuk diye bir tanımlama olacak!
struct Anne {
    std::shared_ptr<Cocuk> cocukPtr;
    ~Anne() { std::cout << "Anne yok edildi.\n"; }
};
struct Cocuk {
    std::shared_ptr<Anne> annePtr;
    ~Cocuk() { std::cout << "Cocuk yok edildi.\n"; }
};
void hataOlustur() {
    auto anne = std::make_shared<Anne>();
    auto cocuk = std::make_shared<Cocuk>();

    anne->cocukPtr = cocuk; // Anne çocuğu sahipleniyor (Sayaç: 1)
    cocuk->annePtr = anne; // Çocuk anneyi sahipleniyor (Sayaç: 1)
} // Fonksiyon biterken sayaçlar 0'a inmeliydi ama birbirlerini tuttukları için 1'de kalırlar!
→ Bellek sızıntısı olur.
```

`hataOlustur()` fonksiyonu bittiğinde ekranda hiçbir "yok edildi" yazısı görmezsiniz ancak bellek sızıntısı olmuştur. `weak_ptr`, bir nesneyi "gözlemler" ama onun sahibi olmaz (referans sayacını artırmaz). Döngüyü kırmak için çocuk nesnesinin annesini `weak_ptr` ile tutması gerekir.

```
#include <iostream>
#include <memory>
struct Cocuk;
struct Anne {
    std::shared_ptr<Cocuk> cocukPtr;
    ~Anne() { std::cout << "Anne yok edildi.\n"; }
};
struct Cocuk {
    std::weak_ptr<Anne> annePtr; // shared_ptr yerine weak_ptr kullandık!
```

```

~Cocuk() { std::cout << "Cocuk yok edildi.\n"; }
void anneminAdiniDeSoyle() {
    // weak_ptr doğrudan kullanılamaz, önce shared_ptr'a dönüştürülmeli (lock)
    if (auto sPtr = annePtr.lock()) {
        std::cout << "Annem hala yasiyor.\n";
    } else {
        std::cout << "Annem bellekten silinmis.\n";
    }
}
};
int main() {
    {
        auto anne = std::make_shared<Anne>();
        auto cocuk = std::make_shared<Cocuk>();

        anne->cocukPtr = cocuk;
        cocuk->annePtr = anne; // weak_ptr olduğu için Anne'nin sayacı artmaz!
    } // Kapsam biterken Anne silinir, Anne silindiği için Cocuk da silinir.
    std::cout << "Program sonu.\n";
}
/*Program Çıktısı:
Anne yok edildi.
Cocuk yok edildi.
Program sonu.
*/

```

Örnek kodlarda akıllı gösterici imal etmekte kullanılan `make_` fonksiyonlarının kullanımı çok önemlidir. Akıllı gösterici oluştururken doğrudan `new` kullanmak yerine `std::make_unique` ve `std::make_shared` kullanılması önerilir. Nesne oluşturulurken istisna (`exception`) oluşursa bellek sızıntısını önler. `make_shared`, nesne ve sayaç bloğu için tek bir bellek tahsisi yaparak daha hızlı çalışır.

§15.7.5 Akıllı Gösterici Örnekleri

9.3 Başlığında dinamik değişken kullanımını anlatan eski tip kullanım örneğini aşağıdaki gibi yenileyebiliriz;

```

#include <iostream>
#include <memory> // std::unique_ptr ve std::make_unique için şart
int main() {
    // 1. TEKİL DEĞİŞKEN (Modern Yöntem)
    // make_unique hem belleği tahsis eder hem de değeri içine koyar.
    std::unique_ptr<double> dPtr = std::make_unique<double>(3.14);
    std::cout << "Double Deger: " << *dPtr << " | Adres: " << dPtr.get() << std::endl;

    // 2. TEKİL DEĞİŞKEN (Farklı ilklendirme)
    auto dPtr2 = std::make_unique<double>(3.1415);
    std::cout << "Double Deger 2: " << *dPtr2 << " | Adres: " << dPtr2.get() << std::endl;
    // NOT: Burada 'delete dPtr' yazmıyoruz! Kapsam (main) bitince otomatik silinirler.
    // 3. DİNAMİK DİZİ (Modern Yöntem)
    int boyut = 5;
    // unique_ptr dizi olduğunu <double[]> ifadesinden anlar.
    std::unique_ptr<double[]> diziPtr = std::make_unique<double[]>(boyut);

    diziPtr[0] = 10.5;
    diziPtr[1] = 20.7;
    diziPtr[2] = 30.9;
    // Atanmayan 3. ve 4. indisler varsayılan olarak 0.0 ile doldurulur (make_unique özelliği).
    std::cout << "\n--- Akıllı Dizi Elemanlari ---" << std::endl;
    for (int i = 0; i < boyut; i++) {
        std::cout << i << ". eleman: " << diziPtr[i] << " | Adres: " << &diziPtr[i] << std::endl;
    }
}

```

```

// KRİTİK: delete[] yazmanıza gerek yok. unique_ptr dizi olduğunu bildiği için
// arka planda doğru silme işlemini (delete[]) kendisi yapar.
}
/*Program Çıktısı:
Double Deger: 3.14 | Adres: 0x2712e2b0
Double Deger 2: 3.1415 | Adres: 0x2712e6e0

--- Akıllı Dizi Elemanlari ---
0. eleman: 10.5 | Adres: 0x2712e700
1. eleman: 20.7 | Adres: 0x2712e708
2. eleman: 30.9 | Adres: 0x2712e710
3. eleman: 0 | Adres: 0x2712e718
4. eleman: 0 | Adres: 0x2712e720
*/

```

Benzer şekilde 11.5 Başlığında dinamik nesne imalatını anlatan eski tip kullanım örneğini aşağıdaki gibi yenileyebiliriz;

```

#include <iostream>
#include <memory> // std::unique_ptr ve std::make_unique için gerekli
class Karmasik {
public:
    float gercek, sanal;
    // 1. Varsayılan Yapıcı
    Karmasik() : gercek(0.0f), sanal(0.0f) {
        std::cout << "Varsayılan yapıcı cagrildi" << std::endl;
    }
    // 2. Parametrelili Yapıcı
    Karmasik(float pGercek, float pSanal) : gercek(pGercek), sanal(pSanal) {
        std::cout << "İki Parametrelili yapıcı cagrildi" << std::endl;
    }
    // 3. Devredilen Yapıcı (Delegating Constructor)
    Karmasik(float pGercek) : Karmasik(pGercek, 0.0f) {
        std::cout << "Tek parametrelili yapıcı cagrildi (İki parametreliliye devredildi)" <<
            std::endl;
    }
    // Yıkıcı (Destructor): Belleğin ne zaman silindiğini görmek için ekledik
    ~Karmasik() {
        std::cout << "Nesne bellekten siliniyor: " << gercek << " + " << sanal << "i" <<
            std::endl;
    }
    void yaz() const {
        std::cout << gercek << " + " << sanal << "i" << std::endl;
    }
};

int main() {
    std::cout << "--- Nesne 1 (Default) ---" << std::endl;
    // std::make_unique parametresiz çağrıldığında varsayılan yapıcıyı kullanır.
    auto k1 = std::make_unique<Karmasik>();
    k1->yaz();
    std::cout << "\n--- Nesne 2 (Parametrelili) ---" << std::endl;
    // make_unique içine gönderilen argümanlar uygun yapıcıya iletilir.
    auto k2 = std::make_unique<Karmasik>(2.0f, 3.0f);
    k2->yaz();
    std::cout << "\n--- Nesne 3 (Tek Parametrelili) ---" << std::endl;
    auto k3 = std::make_unique<Karmasik>(4.0f);
    k3->yaz();
    std::cout << "\n--- Program Sonu ---" << std::endl;
    // NOT: Burada 'delete' yazmıyoruz! Program (veya kapsam) sona erdiğinde akıllı işaretçiler
    // otomatik olarak yıkıcıyı (~Karmasik) çağırır.
}
/*Program Çıktısı:
--- Nesne 1 (Default) ---

```

```

Varsayılan yapıcı çağrıldı
0 + 0i

--- Nesne 2 (Parametrelili) ---
İki Parametrelili yapıcı çağrıldı
2 + 3i

--- Nesne 3 (Tek Parametrelili) ---
İki Parametrelili yapıcı çağrıldı
Tek parametrelili yapıcı çağrıldı (İki parametreliliye devredildi)
4 + 0i

--- Program Sonu ---
Nesne bellekten siliniyor: 4 + 0i
Nesne bellekten siliniyor: 2 + 3i
Nesne bellekten siliniyor: 0 + 0i
*/

```

Akıllı göstericiler ile yapılan güvenli kod örneklerine tasarım desenleri başlığında bolca örnek bulabilirsiniz.

15.8 Standart Şablon Kütüphanesi

Standart şablon kütüphanesi STL(**Standart Template Library**), C++ dilinin en güçlü kütüphanesidir. Jenerik programlama (**generic programming**) ile birçok algoritma programcıya bu kütüphane ile sunulur. Şablon (**template**) kavramını anlamışsanız bu kütüphaneyi iyi kavrarsınız.

Standart şablon kütüphanesi, vektörler, listeler, kuyruklar ve yığınlar gibi popüler ve yaygın olarak kullanılan güçlü bir şablon sınıfları kümesidir. Bu kütüphane algoritmaları ve veri yapılarını içeren şablonlarla genel amaçlı sınıflar ve fonksiyonlar sağlar. Bu şablonlar başlıkta incelenebilir;

1. **Konteynerler:** Belirli bir türdeki nesne kümeleri yani koleksiyonlarını yönetmek için kullanılan veri yapıları şablonudur. **deque**, **list**, **vector**, **map** gibi çeşitli farklı konteyner türleri vardır. Konteynerler, tuttukları verilerden bağımsız olarak hazırlanmış şablon şeklinde dinamik veri yapılarıdır (**dynamic data structure**).
2. **Algoritmalar:** Konteynerler üzerinde etki eder. Konteynerlerin içeriklerinin başlatılmasını, sıralanmasını, aranmasını ve dönüştürülmesini gerçekleştireceğiniz araçları sağlarlar.
3. **Yineleyiciler:** Koleksiyonlarının elemanları arasında gezinmek için kullanılır. Gezinme, koleksiyonların kapsayıcılarında veya kapsayıcıların alt kümeleri olabilir.
4. **Fonksiyon Nesneleri:** Bir fonksiyon sanki bir fonksiyonmuş gibi çağrılabilen bir nesnedir. Normal fonksiyonlar gibi çağrılabilir. Bu yetenek, **fonksiyon çağrı işleci** (**function call operator**) **()** yani fonksiyon parantezlerinin aşırı yüklenmesiyle elde edilir. Bu kelimelerin kısaltılmış hali olan **functor** olarak adlandırılırlar.

15.9 Konteyner Şablonları

Konteyner şablonları bize nesne kümelerini depolamak ve yönetmek için kullanılan veri yapılarıdır. Dinamik veri yapıları yapısal programlama sürecinde çokça geliştirilen ve artık standart hale gelen koleksiyonlardır. Üç başlıkta incelenebilirler;

1. **Ardışık konteynerler** (**sequential container**), elemanları belirli bir doğrusal düzende depolarlar. Elemanları ve sırasını ve düzenlemesini yönetmek için işlemlere doğrudan erişim sağlarlar;
 - ◇ **array** (Dizi): Bunlar sabit boyutlu eleman koleksiyonlarıdır.
 - ◇ **vector** (Vektör): Gerekğinde boyutunu değiştirebilen dinamik bir dizidir.
 - ◇ **deque** (İki Uçlu Kuyruk): Her iki uçtan hızlı ekleme ve silmeye izin veren çift uçlu kuyruk.
 - ◇ **list** (Liste): Çift yönlü yinelemeye izin veren çift bağlantılı liste.
 - ◇ **forward_list** (İleri Liste): Verimli ekleme ve silmeye izin veren ancak yalnızca bir yönde gezinmeye izin veren tek bağlantılı listedir.
 - ◇ **string** (Dizgi): Diğer ardışık konteynerlere benzeyen dinamik bir karakter dizisidir.
2. **İlişkili konteynerler** (**associated container**), elemanları anahtarlar (**key**) göre hızlı bir şekilde geri almaya izin veren sıralı bir düzende depolar. Bu konteynerler, her bir elemanın benzersiz bir anahtarla ilişkilendirildiği bir anahtar-değer (**key-value**) çifti yapısında çalışır. Bu anahtar, karşılık gelen değere hızlı erişim için kullanılır;
 - ◇ **set** (Küme): Belirli bir düzende sıralanmış benzersiz elemanların bir koleksiyonudur.
 - ◇ **map** (Harita): Anahtarların benzersiz olduğu anahtar-değer çiftlerinin bir koleksiyonudur.
 - ◇ **multiset** (Çoklu Küme): Bir kümeye benzer, ancak yinelenen elemanların izin verir.

- ◊ **multimap** (Çoklu Harita): Bir haritaya benzer, ancak yinelenen anahtarlara izin verir.
- 3. **Sıralanmamış ilişkisel konteynerler** (unordered associated container), elemanları sırasız bir şekilde depolar ve anahtarlara (key) dayalı olarak verimli erişim, ekleme ve silmeye olanak tanır. Elemanların sıralı bir düzenini korumak yerine verileri düzenlemek için karma (hash) kullanırlar;
 - ◊ **unordered_set** (Sıralanmamış Küme): Belirli bir sırası olmayan, benzersiz elemanlardan oluşan bir koleksiyondur.
 - ◊ **unordered_map** (Sıralanmamış Harita): Belirli bir sırası olmayan, anahtar-değer çiftlerinden oluşan bir koleksiyondur.
 - ◊ **unordered_multiset** (Sıralanmamış Çoklu Küme): Belirli bir sırası olmayan, yinelenen elemanlara izin verir.
 - ◊ **unordered_multimap** (Sıralanmamış Çoklu Harita): Belirli bir sırası olmayan, yinelenen anahtarlara izin verir.

Konteyner adaptörleri (**container adapter**), mevcut konteynerler için farklı bir ara yüz sağlar. Temel konteynerlerin davranışlarını belirli kullanım durumlarına uyacak şekilde adapte ederler;

- ◊ **stack** (Yığın): Son Giren İlk Çıkar LIFO (**last in first out**) ilkesini izleyen bir veri yapısıdır.
- ◊ **queue** (Kuyruk): İlk Giren İlk Çıkar FIFO (**first in first out**) ilkesini izleyen bir veri yapısıdır.
- ◊ **priority_queue** (Öncelikli Kuyruk): Elemanların önceliğe göre kaldırıldığı özel bir kuyruk türüdür.

§15.9.1 std::array

Dizileri, 8 başlığından biliyoruz. Ancak bu şablon her türlü veri tipiyle oluşturulabilir. Bu sınıf şablonunun iki parametresi vardır. **class T**, Dizi elemanlarının veri tipini belirtir. **std::size_t N** ise dizide kaç eleman olduğunu belirtir.

Diziyi Başlatma

Dizinin sayılarla ifade edilen (**scalar**) veri tiplerini barındırması halinde aşağıdaki şekilde ilk değer verilebilir ya da başlatılabilir);

```
//1. Dizilere ilk değer verme yöntemini kullanarak
std::array<int, 3> a { 0, 1, 2 };
// buna eşdeğer ifade:
std::array<int, 3> a = { 0, 1, 2 };
//2.Kopya yapıcı (copy constructor) kullanarak
std::array<int, 3> a { 0, 1, 2 };
std::array<int, 3> a2(a);
// buna eşdeğer ifade
std::array<int, 3> a2 = a;
//3.Tasıma yapıcısı (move constructor) kullanarak;
std::array<int, 3> a = std::array<int, 3>{ 0, 1, 2 };
```

Dizinin sayılarla ifade edilmeyen (**non scalar**) veri tiplerini barındırması halinde aşağıdaki şekilde ilk değer verilebilir ya da başlatılabilir);

```
struct A { int values[3]; };
//4.Tırnaklı parantez ile ilk değer verilebilir
std::array<A, 2> a{ 0, 1, 2, 3, 4, 5 };
// buna eşdeğer ifade:
std::array<A, 2> a = { 0, 1, 2, 3, 4, 5 };
//5.Tırnaklı parantezleri alt elemanlarda kullanarak;
std::array<A, 2> a{ A{ 0, 1, 2 }, A{ 3, 4, 5 } };
// buna eşdeğer ifade:
std::array<A, 2> a = { A{ 0, 1, 2 }, A{ 3, 4, 5 } };
//6.Tamamıyla tırnaklı parantez kullanarak;
std::array<A, 2> a{{ { 0, 1, 2 }, { 3, 4, 5 } }};
// buna eşdeğer ifade:
std::array<A, 2> a = {{ { 0, 1, 2 }, { 3, 4, 5 } }};
//7.Kopya yapıcı (copy constructor) kullanarak;
std::array<A, 2> a{ 1, 2, 3 };
std::array<A, 2> a2(a);
// buna eşdeğer ifade:
std::array<A, 2> a2 = a;
//8.Taşıma yapıcısı (move constructor) kullanarak;
std::array<A, 2> a = std::array<A, 2>{ 0, 1, 2, 3, 4, 5 };
```

Dizi Elemanlarına Erişim

1. `at(pos)` fonksiyonu ile: `pos` konumundaki elemana sınır denetimiyle bir referans döndürür. `pos`, konteynerin aralığında değilse, `std::out_of_range` türünde bir istisna atılır.

```
#include <array>
int main()
{
    std::array<int, 3> arr;
    arr.at(0) = 3; //0. Elemana 2 atandı
    arr.at(1) = 4; //1. Elemana 2 atandı
    arr.at(2) = 6; //2. Elemana 2 atandı
    int a = arr.at(0); // a değişkenine 0. Dizi elemanı atandı
    int b = arr.at(1); // a değişkenine 1. Dizi elemanı atandı
    int c = arr.at(2); // a değişkenine 2. Dizi elemanı atandı
}
```

2. `operator[pos]` işleci ile: `pos` konumundaki elemana sınır denetimi olmadan bir başvuru döndürür. `pos`, konteynerin aralığında değilse, bir çalışma zamanı hatası (**segmentation violation**) oluşabilir. Bu yöntem, klasik dizilere eşdeğer eleman erişimi sağlar ve bu nedenle `at(pos)`'tan daha verimlidir.

```
#include <array>
int main()
{
    std::array<int, 3> arr;
    arr[0] = 3;
    arr[1] = 4;
    arr[2] = 6;
    int a = arr[0];
    int b = arr[1];
    int c = arr[2];
}
```

3. `std::get<pos>` ile: `std::array` sınıfının üyesi olmayan bu fonksiyon, sınır denetimi olmaksızın derleme zamanı sabit konumu `pos`'taki elemana bir referans döndürür. `pos`, konteynerin aralığında değilse, bir çalışma zamanı hatası (**segmentation violation**) oluşabilir.

```
#include <array>
int main()
{
    std::array<int, 3> arr;
    std::get<0>(arr) = 3;
    std::get<1>(arr) = 4;
    std::get<2>(arr) = 6;
    int a = std::get<0>(arr);
    int b = std::get<1>(arr);
    int c = std::get<2>(arr);
}
```

4. `front()` fonksiyonu ile: Konteynerdeki ilk elemana bir referans döndürür. Boş bir konteynerde `front()`'u çağırmak tanımsızdır.

```
#include <array>
int main()
{
    std::array<int, 3> arr{ 2, 4, 6 };
    int a = arr.front();
}
```

5. `back()` fonksiyonu ile: Konteynerdeki son elemana bir referans döndürür. Boş bir konteynerde `back()`'u çağırmak tanımsızdır.

```
#include <array>
int main()
{
    std::array<int, 3> arr{ 2, 4, 6 };
    int a = arr.back();
}
```

6. `data()` fonksiyonu ile: Eleman depolaması olarak hizmet eden altta yatan diziye gösterici döndürür.

```
#include <iostream>
#include <array>
void yaz(const int* p, std::size_t size){
    for (std::size_t i = 0; i < size; ++i)
        std::cout << p[i] << ' ';
}
int main(){
    std::array<int, 3> a { 0, 1, 2 };
    yaz(a.data(), 3);
}
```

Dizi Elemanları Üzerinde Yineleme

```
#include <iostream>
#include <array>
int main() {
    std::array<int, 3> arr = { 1, 2, 3 };
    for (auto i : arr)
        std::cout << i << std::endl;
}
```

Dizi Boyutunu Kontrol Etme

```
#include <iostream>
#include <array>
int main() {
    std::array<int, 3> arr = { 1, 2, 3 };
    std::cout << arr.size() << std::endl;
}
```

Dizi Elemanlarını Tek Seferde Değiştirme

```
#include <iostream>
#include <array>
int main() {
    std::array<int, 3> arr = { 1, 2, 3 };
    arr.fill(100);
}
```

§15.9.2 `std::vector`

Bir vektör, otomatik olarak işlenen depolamaya sahip dinamik bir dizidir. Bir vektördeki elemanlara, bir dizideki elemanlar kadar verimli bir şekilde erişilebilir; avantajı, vektörlerin boyut olarak dinamik olarak değişebilmesidir. Depolama açısından vektör verileri (genellikle) dinamik olarak tahsis edilmiş belleğe yerleştirilir, bu nedenle bazı küçük ek yükler gerektirir; tersine `std::array`, beyan edilen boyuta göre otomatik depolama kullanır ve bu nedenle herhangi bir ek yüke sahip değildir.

Vektörler `std::array` gibi başlatılabilir.

Vektör Elemanlarına Erişim

Vektör elemanlarına `[]` işleci ve `at()` fonksiyonu ile erişebiliriz.

```
#include <iostream>
#include <vector>
int main() {
    std::vector<int> v{ 1, 2, 3 };
    // [] işleci:
    int a = v[1];
    v[1] = 4;
    // at() fonksiyonu:
    int b = v.at(2);
    v.at(2) = 5;
    //int c = v.at(3); // Hata 4. Eleman Yok!
    for (std::size_t i = 0; i < v.size(); ++i) {
        v[i] = 1;
    }
}
```



```

    }
}

```

Ayrıca `front()` ve `back()` fonksiyonları ile de vektör elemanlarına erişilebilir;

```

std::vector<int> v{ 4, 5, 6 };
int a = v.front();
v.front() = 3;
int b = v.back(); // v.back() eşdeğerdir: v[v.size() - 1]
v.back() = 7;

```

Vektöre Eleman Ekleme ve Çıkarma

Vektöre eleman ekleme `push_back()`, vektörden eleman çıkarma `pop_back()` ve boş olduğunu kontrol etme `empty()` fonksiyonları ile yapılabilir;

```

#include <iostream>
#include <vector>
int main ()
{
    std::vector<int> v;
    int toplam (0);
    for (int i=1;i<=10;i++) v.push_back(i); //vektör oluşturulup elemanlar ekleniyor
    while (!v.empty()) //vektörün boş olup olmadığı kontrol edilip ilerleniyor
    {
        toplam += v.back();
        v.pop_back(); //son elemanı vektörden çıkar
    }
    std::cout << "toplam: " << toplam << std::endl;
}

```

Vektör Verisi

Vektörde `data()` yöntemi, `std::vector`'un elemanlarını dahili olarak depolamak için kullandığı ham belleğe bir gösterici döndürür. Bu gösterici, genellikle vektör verilerini 8 başlığındaki C tarzı bir dizi bekleyen eski koda geçirirken kullanılır.

```

std::vector<int> v{ 1, 2, 3, 4 };
int* p = v.data();
*p = 4; // v[0]=4
++p;
*p = 3; // v[1]=3
p[1] = 2;
*(p + 2) = 1;

```

Aşağıda bir diziye benzeyen ancak büyümesi durumunda kendi depolama gereksinimlerini otomatik olarak karşılayan vektör konteynerini kullanan program örneği verilmiştir;

```

#include <iostream>
#include <vector>
int main() {
    std::vector<double> vec; // int değerler tutacak std::vector konteyneri
    int i;
    std::cout << "std::vector Uzunluğu = " << vec.size() << std::endl;
    for(i = 0; i < 5; i++) { // 5 değer vektöre koyuluyor.
        vec.push_back(i*1.5);
    }
    std::cout << "Vektörün Son Uzunluğu = " << vec.size() << std::endl;
    for(i = 0; i < 5; i++) { // vektör elemanlarına erişiliyor
        std::cout << "vec[" << i << "] = " << vec[i] << ", ";
    }
    std::cout << std::endl;
    // yineleyiciler üzerinden vektör elemanlarına erişim:
    std::vector<double>::iterator v = vec.begin();
    while( v != vec.end()) {
        std::cout << *v << ", ";
        v++;
    }
}

```

```

    std::cout << std::endl;
}
/*Program Çıktısı:
std::vector Uzunluğu = 0
Vektörün Son Uzunluğu = 5
vec[0] = 0,vec[1] = 1.5,vec[2] = 3,vec[3] = 4.5,vec[4] = 6,
0,1.5,3,4.5,6,
*/

```

§15.9.3 std::map, std::multimap ve std::pair

Haritalar, benzersiz anahtarlara sahip **anahtar-değer** (**key-value**) çiftleri (**pair**) içeren sıralı bir ilişkisel konteynerlerdir. Anahtarlar, `compare()` karşılaştırma işlevi kullanılarak sıralanır. Arama, kaldırma ve ekleme işlemleri olan `search()`, `insert()`, `delete()` için logaritmik karmaşıklığa sahiptir.

- ♦ `std::map` veya `std::multimap`'ten herhangi birini kullanmak için `<map>` başlık dosyası projeye dahil edilmelidir.
- ♦ `std::map` ve `std::multimap` her ikisi de elemanlarının anahtarların artan sırasına göre sıralanmış halde tutar.
- ♦ `std::multimap` durumunda, aynı anahtarın değerleri için sıralama yapılmaz.
- ♦ `std::map` ve `std::multimap` arasındaki temel fark, `std::map`'in aynı anahtar için yinelenen değerlere izin vermemesidir.

Çiftleri Oluşturma

Çiftler aşağıdaki gibi oluşturulup karşılaştırılabilir;

```

std::pair<int, int> p1 = std::make_pair(1, 2);
std::pair<int, int> p2 = std::make_pair(2, 2);
if (p1 == p2)
    std::cout << "eşit";
else
    std::cout << "eşit değil!";

```

Çiftleri Sıralama

Çiftler bir konteyner içindeyken de küçük işleci ile sıralanabilir;

```

#include <iostream>
#include <utility>
#include <vector>
#include <algorithm>
#include <string>
int main() {
    std::vector<std::pair<int, std::string>> v = { {2, "Ali"}, {2, "Mustafa"}, {1, "İlhan"} };
    std::sort(v.begin(), v.end());
    for(const auto& p: v)
        std::cout << "(" << p.first << ", " << p.second << ") ";
}
/*Program Çıktısı:
(1,İlhan) (2,Ali) (2,Mustafa)
*/

```

Haritalarda Yineleme

Haritalarda elemanlar anahtar değer ikilisidir ve aşağıdaki şekilde yineleyiciler ile erişilir;

```

#include <iostream>
#include <map>
int main() {
    std::map<int, std::string> siralama {
        std::make_pair(2, "ahmet"),
        std::make_pair(1, "ali") };
    siralama[1]="mustafa";
    std::cout << siralama.at(2) << std::endl; // ahmet
    auto it = siralama.begin();
    std::cout << it->first << " : " << it->second << std::endl; // "1 : mustafa"
    it++;
}

```

```

std::cout << it->first << " : " << it->second << std::endl; // "2 : ahmet"
std::map < std::string , int> notlar {
    std::make_pair("ahmet",100),
    std::make_pair("ali",90),
    std::make_pair("mustafa",85)
};
notlar["mustafa"]=95;
std::cout << notlar.at("ali") << std::endl; // 90
auto it2 = notlar.rbegin(); //tersten
std::cout << it2->first << " : " << it2->second << std::endl; // "mustafa: 95"
it2++;
std::cout << it2->first << " : " << it2->second << std::endl; // "ali: 90"
}

```

std::multimap Örneği

```

#include <iostream>
#include <map>
int main() {
    std::multimap < int, std::string > mmp {
        std::make_pair(2, "ahmet"),
        std::make_pair(1, "ali"),
        std::make_pair(2, "mustafa")
    };
    auto it = mmp.begin();
    std::cout << it->first << " : " << it->second << std::endl; //"1 : ali"
    it++;
    std::cout << it->first << " : " << it->second << std::endl; //"2 : ahmet"
    it++;
    std::cout << it->first << " : " << it->second << std::endl; //"2 : mustafa"
    std::map < int, std::string > mp {
        std::make_pair(2, "ahmet"),
        std::make_pair(1, "ali"),
        std::make_pair(2, "mustafa")
        // Aynı anathardan var!
    };
    auto it2 = mp.rbegin(); //tersten
    std::cout << it2->first << " : " << it2->second << std::endl; //"2 : ahmet"
    it2++;
    std::cout << it2->first << " : " << it2->second << std::endl; //"1 : ali"
}

```

Haritaya Çift Ekleme

Haritaya anahtar değerler aşağıdaki şekilde eklenebilir;

```

std::map< std::string, size_t > meyvesayisi;
meyvesayisi.insert({"üzüm", 20});
meyvesayisi.insert(make_pair("portakal", 30));
meyvesayisi.insert(pair<std::string, size_t>("muz", 40));
meyvesayisi.insert(map<std::string, size_t>::value_type("çilek", 50));

```

`insert()` fonksiyonu bir yineleyici ve bir `bool` değerden oluşan bir çift döndürür. Döndürülen değer `true` ise haritaya çift eklenmiştir. Değilse eklenmeye çalışılan çift zaten haritada vardır.

```

auto success = meyvesayisi.insert({"üzüm", 20});
if (!success.second) { // zaten üzüm var
    success.first->second += 20; // üzüm miktarı değiştirilebilir
}
meyvesayisi["karpuz"]=10; //haridata olmayan karpuz eklenmiştir.
meyvesayisi.insert({"şeftali", 1}, {"kayısı", 1}, {"limon", 1}, {"mango", 7});

```

Konteynerlere başlatma listeleriyle de eleman eklenebilir;

```

std::map< std::string, size_t > meyvelistesi{ {"zeytin", 0}, {"elma", 0}, {"kavun", 0}};
meyvesayisi.insert(meyvelistesi.begin(), meyvelistesi.end());

```

Haritaya eklenecek çiftler yukarıdaki örneklerde olduğu gibi `make_pair()` ve `emplace()` yöntemleriyle hazırlanabilir;

```
meyvesayisi.emplace("Harnup", 200);
```

Haritalarda bir anahtarın ilk oluşumunun yineleyicisini almak için `find()` fonksiyonu kullanılabilir. Anahtar mevcut değilse `end()` döndürür;

```
std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} };
auto it = mmp.find(6);
if(it!=mmp.end())
    std::cout << it->first << ", " << it->second << std::endl; // 6, 5
else
    std::cout << "Aranan Anahtar Bulunamadı!" << std::endl;
it = mmp.find(66);
if(it!=mmp.end())
    std::cout << it->first << ", " << it->second << std::endl;
else
    std::cout << "Aranan anahtar bulunamadı!" << std::endl;
```

Haritada Çift Arama

Haritalarda bir anahtarın mevcut olduğunu bulmanın bir başka yolu da `count()` fonksiyonunu kullanmaktır. Bu fonksiyon, belirli bir anahtarla ilişkili değer sayısını sayar;

```
std::map< int , int > mp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} };
if(mp.count(3) > 0)
    std::cout << "Anahtar bulundu!" << std::endl;
else
    std::cout << "Anahtar Bulunamadı!" << std::endl;
```

Haritaları Başlatma

Haritaları tanımlama ve ilk değer verme;

```
std::multimap < int, std::string > mmp { std::make_pair(2, "C Lang"),
    std::make_pair(1, "CPP Lang"),
    std::make_pair(2, "Java Lang") };
std::map < int, std::string > mp { std::make_pair(2, "C lang"),
    std::make_pair(1, "CPP Lang"),
    std::make_pair(2, "Java Lang") // Eklenmeyecek!
};
// Yineleyici kullanarak;
std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {6, 8}, {3, 4}, {6, 7} };
auto it = mmp.begin();
std::advance(it,3); //baştan üçüncü çifte git {6, 5}
std::map< int, int > mp(it, mmp.end()); // {6, 5}, {8, 9}, {6, 8}, {3, 4}, {6, 7}
//std::pair dizisi kullanarak;
std::pair< int, int > arr[10];
arr[0] = {1, 3};
arr[1] = {1, 5};
arr[2] = {2, 5};
arr[3] = {0, 1};
std::map< int, int > mp(arr,arr+4); //{0 , 1}, {1, 3}, {2, 5}
//std::pair vektörü kullanarak
std::vector< std::pair<int, int> > v{ {1, 5}, {5, 1}, {3, 6}, {3, 2} };
std::multimap< int, int > mp(v.begin(), v.end()); // {1, 5}, {3, 6}, {3, 2}, {5, 1}
```

Haritaların boş olup olmamasına bağlı olarak `true` veya `false` döndüren bir üye `empty()` fonksiyonuna sahiptir. Ayrıca sahip olunan eleman sayısı da `size()` üye fonksiyonuna da sahiptir.

```
std::map<std::string , int> rank {{"facebook.com", 1} ,{"google.com", 2}, {"youtube.com", 3}};
if(!rank.empty())
    std::cout << "Hatitadadaki eleman sayısı: " << rank.size() << std::endl;
else
    std::cout << "Harita Boş!" << std::endl;
```

Karma Harita ve Düzenlenmemiş Harita

Bir de **karma harita** (**hash map**) vardır. **Düzenlenmemiş harita** (**unordered map**) olarak da adlandırılan bu haritalar, normal bir haritaya benzer şekilde anahtar-değer çiftlerini depolar. Ancak elemanları anahtara göre sıralamaz. Bunun yerine, anahtar için bir **karma** (**hash**) değer, ihtiyaç duyulan anahtar-değer çiftlerine hızlı bir şekilde erişmek için kullanılır;

```
#include <string>
#include <unordered_map>
std::unordered_map<std::string, size_t> meyveSayisi;
```

Haritadan Çift Silme

Haritadaki elemanlar aşağıdaki şekilde silinebilir;

```
std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} };
mmp.clear(); //hepsini sil
auto it = mmp.begin();
std::advance(it,3); // baştan üçüncü elemana git {6, 5}
mmp.erase(it); // {1, 2}, {3, 4}, {8, 9}, {3, 4}, {6, 7}
```

Belli anahtarlara sahip elemanlar **erase()** üye fonksiyonu ile silinebilir;

```
std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} };
mmp.erase(6); // {1, 2}, {3, 4}, {3, 4}, {8, 9}
```

§15.9.4 std::list ve std::forwardlist

Konteynerin herhangi bir yerinden öğelerin hızlı bir şekilde eklenmesini ve kaldırılmasını istiyorsak ileri liste (**std::forward_list**) iyi bir seçimdir. Hızlı rastgele erişim desteklenmez. Bu konteyner, **std::list** ile karşılaştırıldığında, çift yönlü yineleme gerekmediğinde daha fazla alan tasarrufu sağlar.

```
#include <forward_list>
#include <list>
#include <string>
#include <iostream>
template<typename T> std::ostream& operator<<(std::ostream& s, const std::forward_list<T>& v) {
    s << "Forward List";
    s.put('[');
    char comma[3] = {'\0', ' ', '\0'};
    for (const auto& e : v) {
        s << comma << e;
        comma[0] = ',';
    }
    return s << ']';
}
template<typename T> std::ostream& operator<<(std::ostream& s, const std::list<T>& v) {
    s << "List";
    s.put('[');
    char comma[3] = {'\0', ' ', '\0'};
    for (const auto& e : v) {
        s << comma << e;
        comma[0] = ',';
    }
    return s << ']';
}
int main() {
    std::forward_list<std::string> words1 {"İlhan", "Ahmet", "Mehmet", "Mustafa", "Abdullah"};
    std::cout << "İsimler1: " << words1 << '\n';
    std::list<std::string> words2 {"İlhan", "Ahmet", "Mehmet", "Mustafa", "Abdullah"};
    std::cout << "İsimler2: " << words2 << '\n';
}
/* Program Çıktısı:
İsimler1: Forward List[İlhan, Ahmet, Mehmet, Mustafa, Abdullah]
İsimler2: List[İlhan, Ahmet, Mehmet, Mustafa, Abdullah]
*/
```

§15.9.5 std::set ve std::multiset

Küme (`std::set`), elemanları sıralanmış ve benzersiz olan bir tür konteynerdir. Kümede aynı eleman iki defa bulunmaz! Ancak `std::multiset` birden fazla eleman aynı elemana sahip olabilir.

```
#include <iostream>
#include <set>
#include <stdlib.h>
struct custom_compare final
{
    bool operator() (const std::string& left, const std::string& right) const
    {
        int nLeft = atoi(left.c_str());
        int nRight = atoi(right.c_str());
        return nLeft < nRight;
    }
};
int main () {
    std::set<std::string> sut({"1", "2", "5", "23", "6", "290"});
    std::cout << "Ön tanımlı sıralama std::set<std::string> : " << std::endl;
    for (auto &&data: sut)
        std::cout << data << ", ";
    std::set<std::string, custom_compare> sut_custom({"1", "2", "5", "23", "6",
        ↪ "290"}, custom_compare{});
    std::cout << std::endl << "Özel Sıralama : " << std::endl;
    for (auto &&data : sut_custom)
        std::cout << data << ", ";
    auto compare_via_lambda = [](auto &&lhs, auto &&rhs){ return lhs > rhs; };
    using set_via_lambda = std::set<std::string, decltype(compare_via_lambda)>;
    set_via_lambda sut_reverse_via_lambda({"1", "2", "5", "23", "6", "290"}, compare_via_lambda);
    std::cout << std::endl << "Lambda Sıralaması : " << std::endl;
    for (auto &&data : sut_reverse_via_lambda)
        std::cout << data << ", ";
    return 0;
}
/*g++ main.cpp -std=C++14 ile derlenen Program Çıktısı:
Ön tanımlı sıralama std::set<std::string> :
1, 2, 23, 290, 5, 6,
Özel Sıralama :
1, 2, 5, 6, 23, 290,
Lambda Sıralaması :
6, 5, 290, 23, 2, 1,
*/
```

§15.9.6 std::optional

Değeri mevcut olabilen veya olmayabilen bir veri tipi temsil etmek için kullanılan `std::optional` olarak tanımlanan değişkenler isteğe bağlı/belki tipleri olarak da bilinir.

C++17'den önce, `nullptr` değerine sahip göstericilere sahip olmak genellikle bir değer yokluğunu temsil ediyordu. Bu, dinamik olarak tahsis edilmiş ve zaten göstericiler tarafından yönetilen büyük nesneler için iyi bir çözümdür. Ancak, bu çözüm nadiren göstericiler tarafından dinamik olarak tahsis edilen veya yönetilen `int` gibi küçük veya ilkel türler için iyi çalışmaz. Çünkü bu veri tipinde tanımlanan bir değişken her zaman değeri vardır. `std::optional` bu yaygın soruna uygulanabilir bir çözüm sağlar.

```
#include <iostream>
#include <optional>
#include <string>
struct Hayvan {
    std::string adi;
};
struct Person {
    std::string adi;
    std::optional<Hayvan> evcilhayvan;
};
```

```
int main() {
    Person person;
    person.adi = "Ali";
    if (person.evciLhayvan) {
        std::cout << person.adi << " Adlı Kişinin " << person.evciLhayvan->adi
        << "EvciL Hayvanı Var" << std::endl;
    }
    else {
        std::cout << person.adi << " adlı kişinin evciL hayvanı yok!" << std::endl;
    }
}
/* g++ -std=c++17 main.cpp olarak derlenir. Program Çıktısı:
Ali adlı kişinin evciL hayvanı yok!
*/
```

Fonksiyondan geri dönüş değeri olarak da `std::optional` kullanılabilir;

```
std::optional<float> divide(float a, float b) {
    if (b!=0.f) return a/b;
    return {};
}

std::optional olarak tanımlı bir değeri işlemek için value_or() kullanılır;

void print_name( std::ostream& os, std::optional<std::string> const& name ) {
    std::cout << "Name is: " << name.value_or("<name missing>") << '\n';
}
```

§15.9.7 std::function ve std::bind

Herhangi bir fonksiyona kılıf çekmek için kullanılır. C tipi fonksiyon göstericisi kullanmak yerine C++ dilinde `std::function` kullanılır.

```
#include <iostream>
#include <functional>
std::function<void(int , const std::string&)> myFuncObj;
void theFunc(int i, const std::string& s) {
    std::cout << s << ": " << i << std::endl;
}
int main(int argc, char *argv[]) {
    myFuncObj = theFunc;
    myFuncObj(10, "hello world");
}
```

Argümanlarla bir fonksiyonu geri çağırmanın gereken bir durumda `std::bind` ile kullanılan `std::function` aşağıda gösterildiği gibi çok güçlü bir tasarım yapısı verir.

```
#include <iostream>
#include <functional>
class Ocak {
public:
    std::function<void(int, const std::string&)> YumurtaKaynadi = nullptr;
    void YumurtaKaynat() {
        if (YumurtaKaynadi) {
            YumurtaKaynadi(10, "Yumurta Pişti");
        }
    }
};

class Kisi {
public:
    Kisi() {
        auto yumurtaPisinceHaberVer = std::bind(&Kisi::eventHandler, this,
        std::placeholders::_1,
        std::placeholders::_2);
        ocakNesnesi.YumurtaKaynadi = yumurtaPisinceHaberVer;
    }
    void eventHandler(int i, const std::string& s) {
```



```

        std::cout << i << " dakikada " << s << std::endl;
    }
    void OcaktaYumurtaKaynat() {
        ocakNesnesi.YumurtaKaynat();
    }
    Ocak ocakNesnesi;
};
int main(int argc, char *argv[]) {
    Kisi ali;
    ali.OcaktaYumurtaKaynat();
}
/*Program Çıktısı:
10 dakikada Yumurta Pişti
*/

```

Aşağıda `std::function` ile çeşitli fonksiyonların çağrılmasını gösteren bir program verilmiştir;

```

#include <iostream>
#include <functional> //std::function, std::placeholders
#include <vector> //std::vector
double c_function(int x, float y, double z) {
    double res = x + y + z;
    std::cout << "c_function çağrıldı: " << x << "+" << y << "+" << z << "=" << res << std::endl;
    return res;
}
struct yapi
{
    double yapi_function(int x, float y, double z) {
        double res = x + y + z;
        std::cout << "yapi::yapi_function çağrıldı: " << x << "+" << y << "+" << z << "=" << res
        << std::endl;
        return res;
    }
    double farkli_yapi_function(int x, double z, float y, long xx) {
        double res = x + y + z + xx;
        std::cout << "farkli_yapi_function çağrıldı: " << x << "+" << y << "+" << z << "+" << xx
        << "=" << res
        << std::endl;
        return res;
    }
    double operator()(int x, float y, double z) {
        double res = x + y + z;
        std::cout << "yapi::operator() çağrıldı: " << x << "+" << y << "+" << z << "=" << res <<
        << std::endl;
        return res;
    }
};
int main(void) {
    using function_type = std::function<double(int, float, double)>;
    yapi ornek_yapi;
    std::vector<function_type> bindings;
    function_type var1 = c_function;
    bindings.push_back(var1);
    function_type var2 = std::bind(&yapi::yapi_function, ornek_yapi, std::placeholders::_1,
    << std::placeholders::_2, std::placeholders::_3);
    bindings.push_back(var2);
    function_type var3 = std::bind(&yapi::farkli_yapi_function, ornek_yapi, std::placeholders::_1,
    << std::placeholders::_3, std::placeholders::_2, 111);
    bindings.push_back(var3);
    function_type var4 = ornek_yapi; //operator()
    bindings.push_back(var4);
    function_type var5 = [](int x, float y, double z) {
        double res = x + y + z;
    }
}

```

```

        std::cout << "lamda çağrıldı: " << x << "+" << y << "+" << z << "=" << res << std::endl;
        return res;
    };
    bindings.push_back(var5);
    std::cout << "Vektöre saklana fonksiyonlar test ediliyor: x = 1, y = 2, z = 3" << std::endl;
    for (auto f : bindings)
        f(1, 2, 3);
}
/*Program Çıktısı:
Vektöre saklana fonksiyonlar test ediliyor: x = 1, y = 2, z = 3
c_function çağrıldı: 1+2+3=6
yapi::yapi_function çağrıldı: 1+2+3=6
farkli_yapi_function çağrıldı: 1+2+3+11=17
yapi::operator() çağrıldı: 1+2+3=6
lamda çağrıldı: 1+2+3=6
*/

```

§15.9.8 std::tuple

Farklı tiplerdeki değerler de dahil olmak üzere herhangi bir sayıda değeri tek bir dönüş nesnesine toplamak için `std::tuple` kullanılır.

```

içerik...
std::tuple<int, int, int, int> foo(int a, int b) { // or auto (C++14)
    return std::make_tuple(a + b, a - b, a * b, a / b);
    /*dört farklı int değeri geri döndürülüyor*/
}

```

C++ 2017 uyarlamasından sonra tırnaklı parantez olan başlatma listesi kullanılır;

```

std::tuple<int, int, int, int> foo(int a, int b) {
    return {a + b, a - b, a * b, a / b};
}

```

Döndürülen tuple'dan değerleri almak zahmetlidir ve `std::get` şablon fonksiyonunun kullanımını gerektirir:

```

auto mrvs = foo(5, 12);
auto add = std::get<0>(mrvs);
auto sub = std::get<1>(mrvs);
auto mul = std::get<2>(mrvs);
auto div = std::get<3>(mrvs);

```

Eğer tipler fonksiyon dönmeyen önce bildirilebiliyorsa, o zaman `std::tie` bir tuple'ı mevcut değişkenlere açmak için kullanılabilir:

```

int add, sub, mul, div;
std::tie(add, sub, mul, div) = foo(5, 12);

```

`std::tie` ile döndürülen değerlerden birine ihtiyaç duyulmuyorsa `std::ignore` kullanılabilir:

```

std::tie(add, sub, std::ignore, div) = foo(5, 12);

```

Yapılandırılmış bağlamalar `std::tie`'dan kaçınmak için kullanılabilir:

```

auto [add, sub, mul, div] = foo(5,12);

```

§15.9.9 std::string

Metinlerin (`string`) bellekte son karakteri sıfır olan karakter dizileri olarak tutulurlar. Dizgiler, boş bir karakter (`NULL Character`) olan `'\0'` ile sonlanan karakter dizisi olarak saklandığı 9.12 başlığında anlatılmıştı.

Modern C++ dilinde, `std::string` şablonu metinler için kullanılır. C dilinden kalan karakter dizisi yaklaşımını kullanmak çok daha düşük seviyeli ve bellek yönetimi tamamen yazılımcının sorumluluğundadır. C++ `std::string` sınıfı, `std` isim uzayının bir parçasıdır ve 1998'de standartlaştırılmıştır.

Metin Değişkenlerini Başlatma

Metinleri tutan değişkenler aşağıdaki şekilde tanımlanabilir;

```

std::string metin1 = "Merhaba";
std::string metin2("İlhan");

```

```
std::string metin3={'M','e','r','h','a','b','a'}; //Sonunda '\0' karakteri olamadan karakter
→ dizisi
std::string metin4=metin2; //önceden tanımlı bir metinden
std::string metin5(metin3); //önceden tanımlı bir metinden yapıcı ile
```

Metni Parçalara Bölme

Dizgiler belirlenen bir ayraç metni ile belirteç (token) denilen küçük parçalara `strtok()` fonksiyonu ile ayrılabilir;

```
std::string str{ "Pijamalı hasta yağız şoföre çabucak güvendi." };
vector<std::string> tokens;
for (auto i = strtok(&str[0], " "); i != NULL; i = strtok(NULL, " "))
    tokens.push_back(i);
```

Bir `std::string` verilerine `const char*` göstericisi ile erişim sağlamak için `c_str()` üye yöntemi kullanılır;

```
//Göstericiler str nesnesinin altında yatan karakter dizisini gösterir;
std::string str("This is a string.");
const char* cstr = str.c_str(); // cstr göstericisi bu metni gösterir: "This is a string.\0"
const char* data = str.data(); // data göstericisi bu metni gösterir: "This is a string.\0"

//Göstericilerin str nesnesinden ömür boyu bağıni çözmek için kopyasını oluşturun
std::string str("This is a string.");
std::unique_ptr<char []> cstr = std::make_unique<char []>(str.size() + 1);
// Yukarıdaki satırın eşdeğeri: char* cstr_unsafe = new char[str.size() + 1];
std::copy(str.data(), str.data() + str.size(), cstr);
cstr[str.size()] = '\0'; // NULL karakter sonuna eklenir.
// delete[] cstr_unsafe;
std::cout << cstr.get();
```

Buradan da anlaşılacağı üzere `std::string` şablonu arka planında metinler karakter dizileri şeklinde tutulur. `std::string` altında yatan `const char*` üzerinden işlem yapmak yerine C++17 ile birlikte gelen sabit

<code>std::string</code>	Karakter Dizisi
<code>std::string</code> , akış (stream) olarak temsil edilebilen nesneleri tanımlar.	'\0' olarak kodlanan boş karakter (NULL Character), karakterlerden oluşan bir karakter dizisini sonlandırır.
<code>std::string</code> dizi bozulması meydana gelmez çünkü dizeler nesneler olarak temsil edilir.	Bir dizinin tip ve boyut kaybına dizinin bozulması (decay of array) denir. Bu durum genellikle diziyi değer veya gösterici ile fonksiyona geçirdiğimizde ortaya çıkar. Karakter dizisinde böyle bir durum ortaya çıkabilir.
Bir <code>std::string</code> sınıfı, metinleri işlemek için çok sayıda yöntem sağlar.	Karakter dizileri, metinleri işlemek için C++ dilinde yerleşik fonksiyonlar sunmaz.
<code>std::string</code> metinlerine bellek dinamik olarak tahsis edilir.	Karakter dizisine boyutu kadar veri bellekte (data segment) veya yığın bellekte (stack segment) yer statik olarak tahsis edilir.

Tablo 15.2: `std::string` ile Karakter Dizilerinin Karşılaştırması

metin şablonu olan `std::string_view` üzerinden işlemler yapılır. Değiştirilemeyen metin verileri gerektiren işlevler için üstünlük sağlar;

```
//1)Alt metin elde etmek için kopya oluşturarak;
std::string str = "Çoook uzun biiiiiir metiiiiiiin";
//'string::substr' returns a new string (expensive if the string is long)
std::cout << str.substr(15, 10) << '\n';
//2)Hiçbir kopya oluşturmadan;
std::string_view view = str;
// string_view::substr returns a new string_view
std::cout << view.substr(15, 10) << '\n';
```

Geniş Karakterli Metinler

`std::string`, `char` veri tipindeki elemanlarla oluşturulmuştur. `std::wstring` ise uluslararası karakterleri temsil eden `wchar_t` tipindeki geniş karakterler (wide character) oluşturulmuştur. Bu karakterlerden oluşmuş

metin değişmezleri için `L` ön eki kullanılır. Bunlar arasında çeviri yapılabilir;

```
#include <iostream>
#include <string>
#include <codecvt>
#include <locale>
int main() {
    std::locale::global(std::locale(""));
    std::wcout.imbue(std::locale());
    setlocale(LC_ALL, "Turkish");
    const wchar_t message_turkish[] = L"İÜÜĞİİŞŞÇÖÖ"; //Türkçe Karakterler
    //std::wcout << message_turkish << std::endl;
    std::string input_str = "İlhan-this is a -string-";
    std::wstring input_wstr = L"İlhan-this is a -wide- string";
    // conversion
    std::wstring str_turned_to_wstr =
        std::wstring_convert<std::codecvt_utf8<wchar_t>>().from_bytes(input_str);
    std::wcout << str_turned_to_wstr << std::endl;
    std::string wstr_turned_to_str =
        std::wstring_convert<std::codecvt_utf8<wchar_t>>().to_bytes(input_wstr);
    std::cout << wstr_turned_to_str << std::endl;
    wstr_turned_to_str =
        std::wstring_convert<std::codecvt_utf8<wchar_t>>().to_bytes(message_turkish);
    std::cout << wstr_turned_to_str << std::endl;
}
```

Metni Oluşturan Karakter Dizisine Erişme

Dizgenin altında yatan karakter dizisine `[]` işleci ile `at()`, `front()` ve `back()` üye fonksiyonları ile erişilebilir;

```
std::string str("Hello world!");
char c1 = str[6]; // 'w'
char c2 = str.at(7); // 'o'
char c3 = str.front(); // 'H'
char c4 = str.back(); // '!'
```

Dizginin altında yatan karakter dizisi üzerinde yineleme yapılabilir;

```
std::string str = "Hello World!";
//1) Foreach
for (auto c : str)
    std::cout << c;
//2) Geleneksel
for (std::size_t i = 0; i < str.length(); ++i)
    std::cout << str[i];
```

Metni Diğer Veri Tiplerine Dönüştürme

Metinleri diğer veri tiplerine de dönüştürebiliriz;

```
std::string ten = "10";
int num1 = std::stoi(ten);
long num2 = std::stol(ten);
long long num3 = std::stoll(ten);
float num4 = std::stof(ten);
double num5 = std::stod(ten);
long double num6 = std::stold(ten);
```

Metinleri dönüştürürken sayı tabanları da kullanılabilir;

```
std::string ten = "10";
std::string ten_octal = "12";
std::string ten_hex = "0xA";
int num1 = std::stoi(ten, 0, 2); // Returns 2
int num2 = std::stoi(ten_octal, 0, 8); // Returns 10
long num3 = std::stol(ten_hex, 0, 16); // Returns 10
long num4 = std::stol(ten_hex); // Returns 0
```

```
long num5 = std::stol(ten_hex, 0, 0); // Returns 10 as it detects the leading 0x
    → işlemleri birleştirilebilir;
std::string hello = "Hello";
std::string world = "world";
std::string helloworld = hello + world; // "Helloworld"
hello += world; // "Helloworld"
```

Metnin Sonuna Yeni Metin Ekleme

Metinler birinin sonuna diğeri gelecek şekilde `append()` üye fonksiyonu ile birleştirilebilir;

```
std::string app = "test and ";
app.append("test"); // "test and test"
```

Metnin İçinde Karakter Arama

Bir karakteri veya başka bir dizeyi bulmak için `find()` kullanabilirsiniz. İlk eşleşmenin ilk karakterinin konumunu döndürür. Eşleşme bulunamazsa, `std::string::npos` döndürür. `find_first_of()` ile karakterlerin ilk oluşumu bulunur, `find_first_not_of()` ile karakterlerin ilk yokluğunu bulunur, `find_last_of()` ile karakterlerin son oluşumu bulunur, `find_last_not_of()` ile karakterlerin son yokluğu bulunur;

```
std::string str = "Hello world!";
auto it = str.find("world");
if (it != std::string::npos)
    std::cout << "Found at position: " << it << '\n';
else
    std::cout << "Not found!\n";
```

Metin İşleme Fonksiyonları

C++ dilinde metinleri kopyalamak ve birleştirmek için `strcpy()` ve `strcat()` fonksiyonları gibi metin işleme için kullanılan bazı yerleşik yöntemlere sahiptir (Tablo 15.3).

Fonksiyonlar	Açıklama
<code>length()</code>	Metnin karakter uzunluğunu döndürür.
<code>swap()</code>	İki metni yer değiştirmek için kullanılır.
<code>size()</code>	Metnin boyutunu bulmak için kullanılır. Metne bellekte kaç karakter yer ayrılmış onu döner.
<code>resize()</code>	Metnin uzunluğunu belirtilen karakter sayısına kadar yeniden boyutlandırmak için kullanılır.
<code>find()</code>	Metnin içinde parametre olarak geçirilen bir başka metni arar.
<code>push_back()</code>	Parametre olarak geçirilen karakteri metnin sonuna eklemek kullanılır.
<code>pop_back()</code>	Metinden son karakteri çıkarmak için kullanılır.
<code>clear()</code>	Metnin tüm karakterlerini silmek için kullanılır.
<code>strncmp()</code>	Metin ile parametre olarak geçirilen bir başka metnin en fazla ilk <code>num</code> kadar karakterini karşılaştırır.
<code>strcpy()</code>	Kaynak metni hedef metin değişkenine kopyalar
<code>strncpy()</code>	<code>strcpy()</code> fonksiyonuna benzerdir, ancak en fazla <code>n</code> bayt kopyalanır.
<code>strchr()</code>	Metindeki bir karakterin son bulunduğu yeri bulur.
<code>strcat()</code>	Kaynak metnin bir kopyasını hedef metnin sonuna ekler.
<code>find()</code>	Bir metnin içerisinde belirli bir alt metni aramak için kullanılır ve alt metnin ilk karakterinin konumunu döndürür.
<code>replace()</code>	<code>first-last</code> aralığındaki eski değere eşit olan her bir elemanı yeni değere değiştirmek için kullanılır
<code>substr()</code>	Verilen bir metinden bir alt metin oluşturmak için kullanılır.
<code>compare()</code>	İki metni karşılaştırmak ve sonucu tam sayı biçiminde döndürmek için kullanılır.
<code>erase()</code>	Bu fonksiyon bir metnin belli bir kısmını silmek için kullanılır.
<code>rfind()</code>	Metnin son bulunduğu konumu bulmak için kullanılır.
<code>capacity()</code>	Bu fonksiyon, derleyici tarafından dizgiye tahsis edilen kapasiteyi döndürür.
<code>shrink_to_fit()</code>	Bu fonksiyon dizginin kapasitesini en az (minimum) olacak şekilde azaltır.

Tablo 15.3: Metin İşleme Yöntemleri

Bunların yanında metni tutan metin değişkeni kullanılarak metin karakterleri üzerinde işlem yapmak için

yineleme (**iteration**) yöntemlerini kullanabiliriz. Yineleme Şablonları altında örneği verilmiştir.

Klavyeden Bir Satır Metin Okuma

Bir girdi nesnesinden metin okumak için en yaygın yol, C++ dininde akıştan veri ayıklama işleci (**extraction operator**) olan (**>>**) ile **std::cin** nesnesini kullanmaktır.

```
#include <iostream>
#include <string>
int main() {
    std::string metin;
    std::cout << "Bir metin Giriniz: ";
    std::cin >> metin; //sadece ilk sözcük metin değişkenine atanır!
    std::cout<< "Girilen Metin: " << metin << std::endl;
}
/* Program Çalıştığında:
Bir metin Giriniz: İlhan Ozkan
Girilen Metin: İlhan
*/
```

Program incelendiğinde girilen metin iki sözcükten okunmasına rağmen ilk sözcük metin değişkenine atanmıştır. Eğer girilen tüm satırı bu metne aktarmak istersek **<string>** başlık dosyasındaki **getline()** fonksiyonunu kullanmalıyız;

```
#include <iostream>
#include <string>
int main() {
    std::string metin;
    std::cout << "Bir metin Giriniz: ";
    getline(std::cin, metin); // Satır sonu karakteri girilene kadarki metin değişkene atanır!
    std::cout<< "Girilen Metin: " << metin << std::endl;
}
/* Program Çalıştığında:
Bir metin Giriniz: İlhan Özkan
Girilen Metin: İlhan Özkan
*/
```

Klavyeden Geniş Karakterli Metin Okuma

Uluslararası karakterlerle çalışılan aşağıdaki örnek verilebilir. Bu kodu derlemede hata almaya devam ediyorsak linux üzerinde gerekli dil paketlerini **sudo locale-gen tr_TR.UTF-8** ve **sudo update-locale** komutlarını art arda vererek kurmalıyız.

```
#include <iostream>
#include <string>
int main() {
    std::wstring message_turkish = L"ıİüÜğĞiİşŞçÇöÖ"; //Türkçe "Diğer Karakterler
    std::locale::global(std::locale(""));
    std::wcout.imbue(std::locale());
    setlocale(LC_ALL, "Turkish");
    std::wcout << message_turkish << std::endl;
    wchar_t message[100];
    std::wcout << L"Türkçe Bir Metin Giriniz:";
    std::wcin.getline(message,100);
    std::wcout << "Girilen metin:" << message <<std::endl;
    std::wstring message2;
    std::wcout << L"Türkçe Bir Başka Giriniz:";
    getline(std::wcin, message2);
    std::wcout << L"Girilen metin:" << message2;
}
```

15.10 Algoritma Şablonları

Standart Şablon Kütüphanesi'ndeki algoritmalar (**algorithm**), konteynerler üzerinde işlem yapmak için özel olarak tasarlanmış büyük bir fonksiyon kümesidir. Bu fonksiyonlar, konteynerlerin iç yapılarını bilmeye gerek kalmadan konteynerler arasında geçiş yapmak için kullanılan yineleyiciler (**iterator**) kullanılarak uygulanır.

Sıra değiştirmeyen algoritmalar;

- ◊ `for_each`: Bir aralıktaki her bir elemanı bulmak için bir fonksiyonda kullanılır.
- ◊ `count`: Bir aralıktaki değerin oluşum sayısını sayar.
- ◊ `find`: Bir aralıktaki değerin ilk oluşumunu bulur.
- ◊ `find_if`: Bir yordamı karşılayan ilk elemanı bulur.
- ◊ `find_if_not`: Bir yordamı karşılamayan ilk elemanı bulur.
- ◊ `equal`: İki aralığın eşit olup olmadığını kontrol eder.
- ◊ `search`: Bir dizi içinde bir alt diziyi arar.

Sıra değiştiren algoritmalar;

- ◊ `copy`: Elemanları bir aralıktan diğerine kopyalar.
- ◊ `copy_if`: Bir yordamı karşılayan elemanları başka bir aralığa kopyalar.
- ◊ `copy_n`: Belirli sayıda elemanı bir aralıktan diğerine kopyalar.
- ◊ `move`: Elemanları bir aralıktan diğerine taşır.
- ◊ `transform`: Bir aralığa bir fonksiyon uygular ve sonucu depolar.
- ◊ `remove`: Belirli bir değere sahip elemanları bir aralıktan kaldırır.
- ◊ `remove_if`: Bir yordamı karşılayan elemanları kaldırır.
- ◊ `unique`: Ardışık yinelenen elemanları kaldırır.
- ◊ `reverse`: Bir aralıktaki elemanların sırasını tersine çevirir.
- ◊ `swap`: Elemanları değiştirir.

Sıralama algoritmaları;

- ◊ `sort`: Bir aralıktaki elemanları sıralar.
- ◊ `stable_sort`: Eşdeğer elemanların göreceli sırasını koruyarak elemanları sıralar.
- ◊ `partial_sort`: Bir aralığın bir kısmını sıralar.
- ◊ `nth_element`: Aralığı, n'inci elemanın son konumunda olacak şekilde bölümlere ayırır.

Arama algoritmaları;

- ◊ `binary_search`: Sıralanmış bir aralıkta bir elemanın bulunup bulunmadığını kontrol eder.
- ◊ `lower_bound`: Sırayı korumak için bir elemanın eklenebileceği ilk konumu arar.
- ◊ `upper_bound`: Belirtilen değerden büyük olan ilk elemanın konumunu arar.
- ◊ `equal_range`: Eşit elemanların aralığını döndürür.

Öbek (**heap**) algoritmaları;

- ◊ `make_heap`: Bir aralıktan bir öbek oluşturur.
- ◊ `push_heap`: Bir öbeğe bir eleman ekler.
- ◊ `pop_heap`: Bir öbekten en büyük eleman kaldırır.
- ◊ `sort_heap`: Bir öbekteki elemanları sıralar.

Küme algoritmaları;

- ◊ `set_union`: İki kümenin birleşimini hesaplar.
- ◊ `set_intersection`: İki kümenin kesişimini hesaplar.
- ◊ `set_difference`: İki küme arasındaki farkı hesaplar.
- ◊ `set_symmetric_difference`: İki küme arasındaki simetrik farkı hesaplar.

Sayısal algoritmalar;

- ◊ `accumulate`: bir aralığın toplamını (veya diğer işlemleri) hesaplar.
- ◊ `inner_product`: İki aralığın iç çarpımını hesaplar.
- ◊ `adjacent_difference`: Bitişik elemanlar arasındaki farkları hesaplar.
- ◊ `partial_sum`: bir aralığın kısmi toplamalarını hesaplar.

Diğer algoritmalar;

- ◊ `generate`: Bir fonksiyon tarafından üretilen değerlerle bir aralığı doldurur.
- ◊ `shuffle`: Bir aralıktaki elemanları rastgele karıştırır.
- ◊ `partition`: Elemanları bir öngörüye göre iki gruba ayırır.

Bu algoritmalar genellikle `<algorithm>` başlığı altında toplanır ve yineleyiciler (**iterator**) aracılığıyla veri yapıları (`vector`, `list`, `array`) üzerinde işlem yapar. Aşağıda birkaçına örnek verilmiştir;

1. `std::sort` (Sıralama Algoritması) :En sık kullanılan şablonlardan biridir. Verilen aralıktaki elemanları (varsayılan olarak küçükten büyüğe) sıralar.

```
#include <iostream>
#include <vector>
#include <algorithm> // Algoritma şablonları için
```



```
int main() {
    std::vector<int> sayilar = {50, 10, 40, 20, 30};
    // Şablon her türlü karşılaştırılabilir tip için çalışır
    std::sort(sayilar.begin(), sayilar.end());
    for(int n : sayilar)
        std::cout << n << " "; // Çıktı: 10 20 30 40 50
}
```

2. **std::find** (Arama Algoritması): Bir veri kümesi içinde belirli bir değeri arar. Eğer değeri bulursa o elemana ait bir yineleyici, bulamazsa sonu (**end()**) döndürür.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    std::vector<std::string> isimler = {"Ali", "Veli", "Ayse"};
    // "Veli" ismini arıyoruz
    auto it = std::find(isimler.begin(), isimler.end(), "Veli");
    if (it != isimler.end()) {
        std::cout << "Bulundu: " << *it << std::endl;
    } else {
        std::cout << "Bulunamadi!" << std::endl;
    }
}
```

3. **std::for_each** (Her Eleman İçin İşlem): Belirli bir aralıktaki her elemana, sizin belirlediğiniz bir fonksiyonu veya lamda ifadesini uygular.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};
    // Her elemanın karesini ekrana yazdıran bir lambda şablonu
    std::for_each(v.begin(), v.end(), [](int &n) {
        n *= n;
        std::cout << n << " ";
    }); // Çıktı: 1 4 9 16 25
}
```

4. **std::accumulate** (Biriktirme/Toplam Algoritması): Bir aralıktaki tüm değerleri toplamak (veya başka bir işleme sokmak) için kullanılır. **<numeric>** başlığı altında bulunur.

```
#include <iostream>
#include <vector>
#include <numeric> // accumulate için gerekli
int main() {
    std::vector<double> fiyatlar = {10.5, 20.0, 5.25};
    // 0.0 başlangıç değeri üzerine tüm elemanları ekler
    double toplam = std::accumulate(fiyatlar.begin(), fiyatlar.end(), 0.0);
    std::cout << "Toplam Fiyat: " << toplam << std::endl; // 35.75
}
```

15.11 Yineleme Şablonları

Standart Şablon Kütüphanesi'ndeki yineleyiciler (**iterator**), bir konteyner içindeki elemanlara gösterici görevi gören ve verilere erişmek ve bunları düzenlemek için tekdüze bir ara yüz sağlayan nesnelerdir. Algoritmalar ve konteynerler arasında bir köprü görevi görürler;

- ◊ Giriş Yineleyicileri (**input iterator**): Elemanlara salt okunur erişime izin verirler.
- ◊ Çıkış Yineleyicileri (**output iterator**): Elemanlara salt yazılır erişime izin verirler.
- ◊ İleri Yineleyiciler (**forward iterator**): Artırılabilen okuma ve yazmaya izin verirler.
- ◊ Çift Yönlü Yineleyiciler (**bidirectional iterators**): Artırılabilir ve azaltılabilirler.
- ◊ Rastgele Erişim Yineleyicileri (**random access iterator**): Aritmetik işlemleri desteklerler ve elemanlara doğrudan erişebilirler.

Yineleme nesneleri, diziler gibi birden çok aynı nesneyi tutan veri yapılarında elemanlar üzerinde hareket

etmeyi sağlayan nesnelerdir.

Fonksiyon	Açıklama
<code>begin()</code>	Bu fonksiyon, dizginin başlangıcına işaret eden bir yineleyici (<code>iterator</code>) nesne döndürür.
<code>end()</code>	Bu fonksiyon, dizginin sonunu işaret eden bir yineleyici nesne döndürür.
<code>rbegin()</code>	Bu fonksiyon, dizginin başlangıcına işaret eden ters yineleyici (<code>reverse_iterator</code>) nesne döndürür.
<code>rend()</code>	Bu fonksiyon, dizginin sonunu işaret eden ters yineleyici nesne döndürür.
<code>cbegin()</code>	Bu fonksiyon, dizginin başlangıcına işaret eden bir sabit yineleyici (<code>const_iterator</code>) döndürür.
<code>cend()</code>	Bu fonksiyon dizginin sonunu işaret eden bir sabit yineleyici (<code>const_iterator</code>) döndürür.
<code>crbegin()</code>	Bu fonksiyon dizginin sonunu işaret eden bir sabit bir ters yineleyici (<code>const_reverse_iterator</code>) döndürür.
<code>crend()</code>	Bu fonksiyon, dizginin başlangıcına işaret eden bir ters yineleyici (<code>const_reverse_iterator</code>) döndürür.

Tablo 15.4: Yineleme Fonksiyonları

Yineleme şablonlarına ilişkin örnek aşağıda verilmiştir;

```
#include <iostream>
#include <string>
int main() {
    std::string::iterator itr; //string yineleme nesnesi
    std::string::reverse_iterator rit; //string ters yineleme nesnesi
    std::string metin = "Merhana Ilhan.";
    itr = metin.begin();
    std::cout << "Yineleyicinin işaret ettiği karakter: " << *itr<< std::endl;
    itr = metin.end() - 1;
    std::cout << "Yineleyicinin işaret ettiği son metin karakteri: " << *itr << std::endl;
    rit = metin.rbegin();
    std::cout << "Ters yineleyicinin işaret ettiği karakter:: " << *rit << std::endl;
    rit = metin.rend() - 1;
    std::cout << "Ters yineleyicinin işaret ettiği on metin karakteri:: " << *rit << std::endl;
}
/*Program çalıştığında:
Yineleyicinin işaret ettiği karakter: M
Yineleyicinin işaret ettiği son metin karakteri: .
Ters yineleyicinin işaret ettiği karakter:: .
Ters yineleyicinin işaret ettiği on metin karakteri:: M
*/
```

15.12 Fonksiyon Nesneleri

Fonksiyon nesnesi (**function object**), durum saklayabilen ve aşırı yüklenmiş fonksiyon işlecine (**function operator**) () sahip nesnelerdir. Bu nesnelere kısaltma olarak **functor** adı verilir. Bu işleci aşırı yüklemiş nesneler, bir fonksiyonmuş gibi çağrılabilirler ve bir fonksiyon veya fonksiyon göstericisiymiş gibi ele alınırlar.

Aşağıda bir dizinin elemanlarını 1 artıran bir dönüşüm programı verilmiştir;

```
#include <bits/stdc++.h> //transform()
int artirim(int x) { return (x+1); }
int main() {
    int dizi[] = {1, 2, 3, 4, 5};
    int adet = sizeof(dizi)/sizeof(dizi[0]);
    /* Aşağıdaki dönüşüm fonksiyonu ile dizi elemanları 1 artırılıyor */
    std::transform(dizi, dizi+adet, dizi, artirim);
    for (int i=0; i<adet; i++)
        std::cout << dizi[i] <<" ";
}
/* Program Çıktısı:
2 3 4 5 6
*/
```

Aşağıda bir fonksiyon nesnesi üzerinden aynı işlemi yapan bir program örneği verilmiştir;

```
#include <bits/stdc++.h> // transform
class artirim// Functor
{
    private:
        int sayi;
    public:
        artirim(int n) : sayi(n) { } //Yapıcı işlenecek parametreyi alır
        /* Aşağıda verilen işleç önyüklemesi ile sayi kadar artırım işlemi yapılacaktır. */
        int operator () (int pMiktar) const {
            return sayi + pMiktar;
        }
};

int main() {
    int dizi[] = {1, 2, 3, 4, 5};
    int adet = sizeof(dizi)/sizeof(dizi[0]);
    int miktar = 5;
    std::transform(dizi, dizi+adet, dizi, artirim(miktar));
    for (int i=0; i<adet; i++)
        std::cout << dizi[i] << " ";
}

/*Program Çıktısı:
6 7 8 9 10
*/
```

Aritmetik functor (**arithmetic functor**), aritmetik operatörler temel matematiksel işlemleri gerçekleştirmek için kullanılır. Toplama (+), Çıkarma (-), Çarpma (*), Bölme (/), Kalan (%), Negatif (-) işleç işlevlerini yerine getirirler. Karşılaştırma fonksiyon nesneleri (**comparision functors**), özellikle konteynerlerde sıralama veya arama yapmak için değerleri karşılaştırmak amacıyla kullanılır. Küçüktür (<), Büyüktür (>), Küçüktür veya Eşittir (<=), Büyük veya Eşit (>=), Eşit (=), Eşit Değil (!=) işleç işlevlerini yerine getirirler. Mantıksal fonksiyon nesneleri (**logical functors**), Boole mantığını içeren senaryolar için mantıksal işlemleri gerçekleştirir. Mantıksal VE (&&), Mantıksal VEYA (||) ve Mantıksal DEĞİL (!) işleç işlevlerini yerine getirirler. Bit düzeyi fonksiyon nesneleri (**bitwise functors**), bit düzeyi işlemleri gerçekleştirir. Bit düzeyi VE (&), VEYA (|) ve ÖZEL VEYA (^) işleç işlevlerini yerine getirirler.

15.13 using Anahtar Kelimesinin Şablonlarda Kullanımı

using anahtar kelimesi, 10.11.4, 11.12 ve 12.9 başlıklarındaki kullanımı yanında **using** anahtar kelimesi, bir şablonun sadece bazı parametrelerini sabitleyip geri kalanını esnek bırakabilirsiniz;

```
template <typename T>
using NumerikSozluk = std::map<std::string, T>;

// Kullanımı:
NumerikSozluk<int> yaslar; // std::map<string, int> yerine
NumerikSozluk<double> notlar; // std::map<string, double> yerine
```

C++14 ile gelen en büyük yeniliklerden biri değişken şablonlarıdır (variable templates). **using** doğrudan bir değişken şablonu olmasa da, tip dönüşümlerinde **using** ile tanımlanmış tiplerin bu yeni değişken şablonlarıyla kullanımı (özellikle **std::enable_if_t** gibi yapılarda) kodun sadeleşmesini sağlamıştır. C++11 uyarlamasında bir tipin **int** olup olmadığını kontrol etmek için uzunca şunu yazardık;

```
typename std::enable_if<std::is_integral<T>::value>::type
```

C++14 ve sonrasında **using** ve değişken şablonları sayesinde bu şuna dönüştü:

```
std::enable_if_t<std::is_integral_v<T>>
```

15.14 Düşün ve Çöz

- ◇ **Düşün:** C++'da dizi boyutunun derleme zamanında bilinmesi zorunluluğu neden vardır? **std::vector** ile sabit boyutlu dizi arasındaki temel farkı bellek yönetimi açısından açıklayın.
- ◇ **Düşün:** Fonksiyon şablonu (**function template**) ile sınıf şablonu (**class template**) arasındaki farkı açıklayın. **std::sort** gibi bir STL algoritmasının şablon mekanizmasına dayandığını gösterin.
- ◇ **Düşün:** Akıllı göstericiler (**std::unique_ptr**, **std::shared_ptr**, **std::weak_ptr**) ham göstericiler (**raw pointer**) göre neden tercih edilmelidir? Her birinin sahiplik (**ownership**) semantiğini açıklayın.
- ◇ **Düşün:** **std::vector** ile **std::array** arasındaki farkları bellek yönetimi, performans ve kullanım ko-

laylığı açısından karşılaştırın. Hangisi hangi durumda tercih edilmelidir?

- ◊ **Çöz:** `std::map` ile `std::unordered_map` arasındaki farkı arama karmaşıklığı ($O(\log n)$ vs $O(1)$) açısından karşılaştırın. Her birinin kullanım senaryosunu örnekleyin.
- ◊ **Çöz:** Değişken sayıda şablon parametresi alan (**variadic template**) bir fonksiyon yazın. `std::tuple` veri yapısının altında yatan şablon mekanizmasını açıklayın.
- ◊ **Düşün:** Fonksiyon nesneleri (**function objects/functors**) lamda ifadelerinden önce neden yaygın kullanılırdı? `std::function` ve `std::bind` ile bu yapıların modern C++'daki yerini açıklayın.
- ◊ **Çöz:** `std::string` ile C-tarzı karakter dizisi (`char[]`) arasındaki temel farkları karşılaştırın. Bir C-dizisini `std::string`'e dönüştürmenin güvenli yolunu gösterin.
- ◊ **Çöz:** `std::getline` neden `std::cin` » yerine metin okumak için daha uygun bir seçimdir? Boşluk içeren bir cümleyi okuyup içindeki kelimeleri ayrıştıran bir program yazın.
- ◊ **Çöz:** `std::string`'in `find()`, `substr()`, `replace()` ve `erase()` fonksiyonlarını kullanarak bir metin içinde belirtilen bir kelimeyi bulup başka bir kelimeyle değiştiren bir fonksiyon yazın.
- ◊ **Çöz:** Dizgi dönüşüm fonksiyonlarını (`std::stoi()`, `std::stod`, `std::to_string()`) kullanarak kullanıcıdan alınan sayı metinlerini aritmetik işlemlerde kullanan bir hesap makinesi tasarlayın.
- ◊ **Düşün:** Karakter göstericileri (`const char*`) ile `std::string` arasındaki güvenlik farkını açıklayın. Tampon taşması (**buffer overflow**) saldırısının neden karakter dizileriyle daha kolay gerçekleştiğini belirtin.